

Benchmarking single- and multi-objective optimization algorithms:

HOW TO MAKE YOUR EXPERIMENTAL DATA MORE VALUABLE

Diederick Vermetten^{1,2}, Jacob de Nobel³, Thomas Bäck³, Carola Doerr^{1,2}

1



2



3



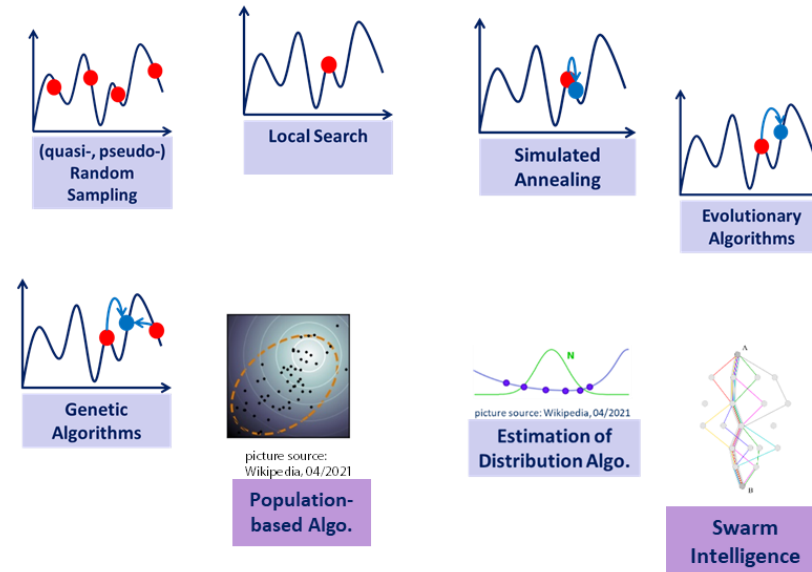
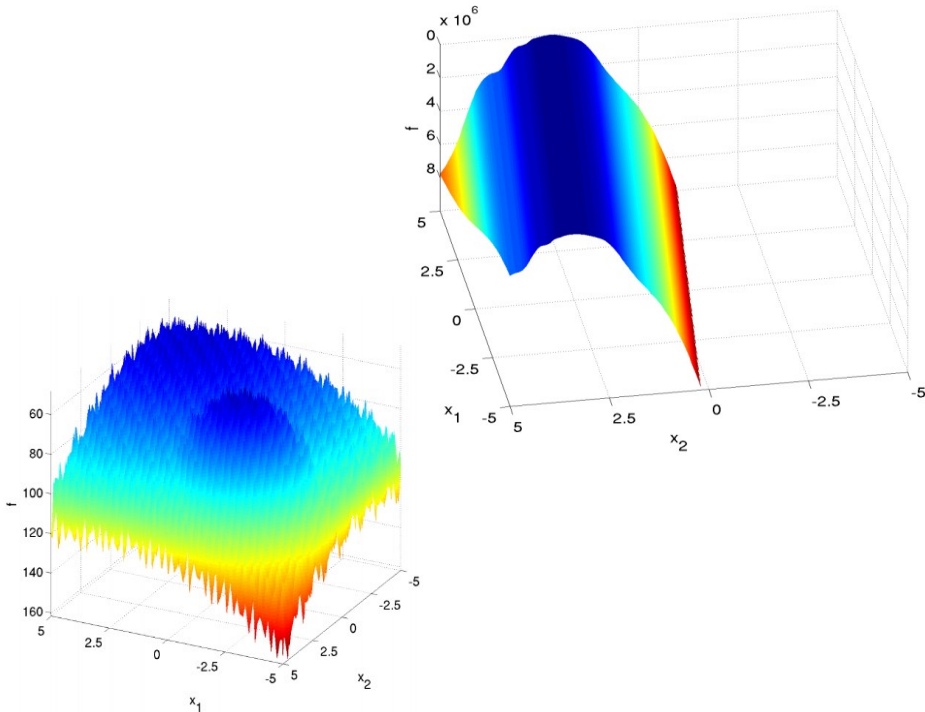
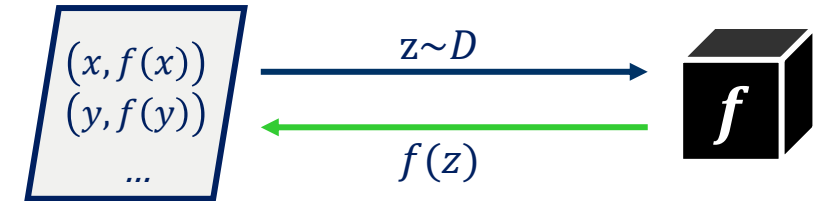
Universiteit
Leiden
The Netherlands

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from Permissions@acm.org.
GECCO '25 Companion, July 14–18, 2025, Malaga, Spain © 2025 Copyright is held by the owner/author(s). Publication rights licensed to ACM. ACM ISBN 979-8-4007-1464-1/25/07...
<https://doi.org/10.1145/3712255.3716544>

Benchmarking Black-Box Optimization

We want to understand the optimization process

Many algorithms, problems



Competitive Benchmarking?

Algorithm A



Algorithm B

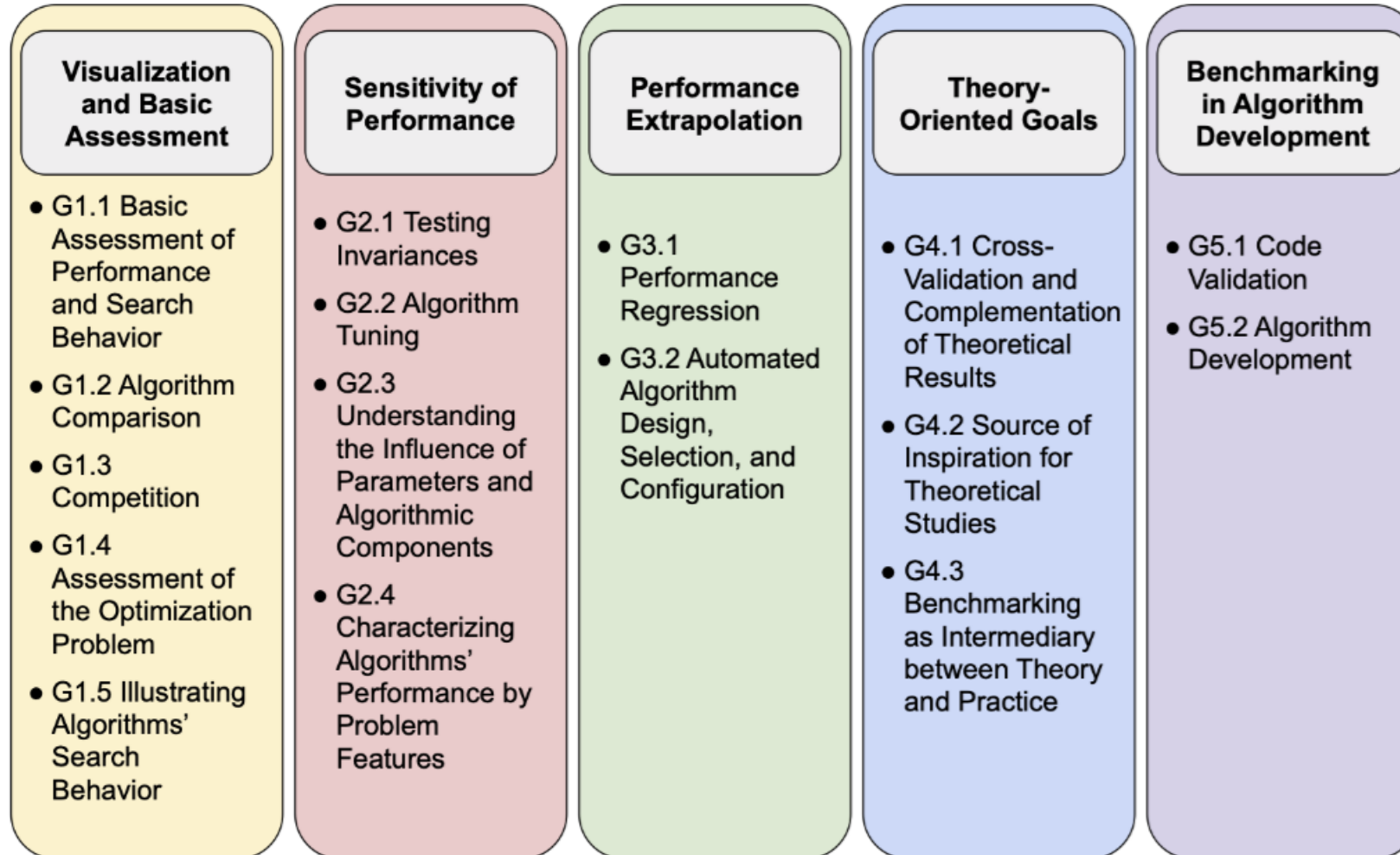
Comparison between algorithms -> Empirical evaluation

Generate data to determine which algorithm is 'better'

'Competitive' approach

Benchmarking can be much more than this!

Goals of Benchmarking



Source: Bartz-Beielstein, T., Doerr, C., Berg, D. V. D., Bossek, J., Chandrasekaran, S., Eftimov, T., ... & Weise, T. (2020). Benchmarking in optimization: Best practice and open issues. *arXiv preprint arXiv:2007.03488*.

Basic Benchmarking Checklist



What is the goal of this benchmarking study?



What algorithm(s) should be included?



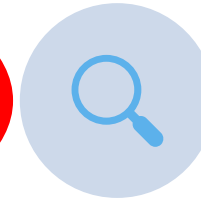
What problem(s) do we use?



How many repetitions to use?



What measure(s) are we interested in?



How to evaluate / visualize the results?



Can we ensure results are reusable / reproducible?

Analyzing Performance Data

Table 1. Comparison results of D²EMO/MGD with the peer algorithms

	n	D ² EMO/MGD	HSMEA	K-RVEA	MOE
ZDT3	3	1.3199(3.34E-3)	1.2914(5.99E-2) [†]	1.1814(1.77E-1) [†]	1.1831
	5	1.3	What do we learn from this?		
	8	1.3			
ZDT31	3	1.3	What do we learn from this?		
	5	1.3			
		8	1.3655(7.06E-2)	1.1851(1.14E-1) [†]	1.1098(1.65E-1) [†]
	3	1.2818(2.18E-2)	1.2309(9.73E-2) [†]	1.1852(7.96E-2) [†]	1.1265
	5	1.2818(2.18E-2)	1.1851(1.14E-1) [†]	1.1098(1.65E-1) [†]	0.9112

Table 4. The *mean* IGD values of IFCS-MOEA/D-DE, FCS-MOEA/D-DE, CPS-MOEA, CSEA, MOEA/D-EGO, and EDN-ARMOEA on UF1–10 and LZ1–9.

		Lots of interesting data underlying these t					
V	UF1	Limited to one perspective, chosen by aut					
	UF2						
	UF3						
	UF4						
W	UF5	4.20e+00 _{3.67e-01}	3.00e+00 _{4.19e-01} (+)	4.49e+00 _{3.64e-01} (-)	2.91e+00 _{5.98e-01} (+)	4.90e+00 _{3.84e-01} (-)	4.53e+00 _{3.80e-01} (-)
	UF6	3.78e+00 _{5.72e-01}	2.48e+00 _{8.10e-01} (+)	4.27e+00 _{8.09e-01} (-)	1.79e+00 _{6.32e-01} (+)	4.31e+00 _{8.82e-01} (-)	4.03e+00 _{7.69e-01} (~)
W	UF7	9.50e-01 _{1.11e-01}	6.03e-01 _{1.51e-01} (+)	1.03e+00 _{1.75e-01} (-)	4.21e-01 _{1.04e-01} (+)	1.08e+00 _{1.88e-01} (-)	1.06e+00 _{2.20e-01} (~)
	UF8	7.24e-01 _{9.32e-02}	1.35e+00 _{3.77e-01} (-)	1.43e+00 _{2.03e-01} (-)	1.40e+00 _{3.25e-01} (-)	1.62e+00 _{3.32e-01} (-)	1.93e+00 _{2.12e-01} (-)
	UF9	7.57e-01 _{8.23e-02}	1.33e+00 _{2.04e-01} (-)	1.38e+00 _{2.92e-01} (-)	1.40e+00 _{2.99e-01} (-)	1.87e+00 _{6.32e-01} (-)	1.81e+00 _{2.13e-01} (-)
W	UF10	4.65e+00 _{4.34e-01}	7.13e+00 _{1.10e+00} (-)	8.06e+00 _{1.20e+00} (-)	7.93e+00 _{1.45e+00} (-)	8.78e+00 _{1.40e+00} (-)	9.72e+00 _{1.25e+00} (-)
	LZ1	1.55e-01 _{1.11e-01}	1.59e-01 _{1.11e-01} (~)	1.52e-01 _{1.11e-01} (~)	1.62e-01 _{1.11e-01} (~)	1.75e-01 _{1.11e-01} (~)	1.61e-01 _{1.11e-01} (~)
	LZ2	Look at data collection and performance analysis differently					
	LZ3						
	LZ4						
	LZ5						
D	LZ6						
	LZ7						
	LZ8						
	LZ9						
+/-/~		9.52e-01 _{1.43e-01}	5.78e-01 _{1.78e-01} (+)	1.01e+00 _{1.37e-01} (~)	4.96e-01 _{1.57e-01} (+)	9.46e-01 _{1.83e-01} (~)	9.50e-01 _{2.18e-01} (~)
			6/9/4	0/16/3	6/11/2	0/15/4	0/12/7
DTLZ2		5 1.4075(2.62E-3)	1.3926(1.00E-2) [†]	1.3313(1.72E-2) [†]	1.1908	10 30 2.2000E+2 = 1.4004E+2 ± 1	
		8 1.4052(2.34E-3)	1.3875(1.20E-1) [†]	1.3170(1.53E-2) [†]	1.0411	+/-/~ ≈ 5/28/3	1/20/15

Table 3. IGD+ performance metrics for 31 runs on WFG problems. N $\tilde$ SGA-III (5-th and 6-th columns) is NSGA-III without the sorting process.

Table 1. The statistical results of the IGD metric values obtained by five algorithms with $D = 30$ on 36 test instances over 30 independent runs.

Learn from this?

			K-RVEA	KTA2	CREMO	
			$2 \approx 5.6634e+2$	$- 4.4598e+2$	$\approx 4.4958e+2$	
			$2 \approx 4.0149e+2$	$- 3.5212e+2$	$- 3.2230e+2$	
			$2 \approx 2.3553e+2$	$\approx 2.6764e+2$	$- 2.2392e+2$	
	10 30	$3.1911e+2$	$- 2.2270e+2$			
	3 30	$1.0722e+0$	$- 8.8657e-1$	$\approx 1.5288e+0$	$- 6.8211e-1$	$+ 8.2374e-1$
DTLZ2	6 30	$1.9690e+0$	$- 1.2255e+0$	$- 1.3797e+0$	$- 1.1949e+0$	$- 9.7444e-1$
				$- 1.2770e+0$	$- 1.0349e+0$	
				$- 1.2840e+3$	$\approx 1.2645e+3$	
				$- 1.3212e+3$	$\approx 1.1896e+3$	
				$- 1.0737e+3$	$- 8.2485e+2$	
				$- 8.4833e-1$	$- 8.1190e-1$	
				$- 9.8425e-1$	$- 1.1409e+0$	
				$- 6.4730e-1$	$- 7.5761e-1$	
				$- 5.6524e-1$	$- 2.1172e+1$	
				$- 1.9871e+1$	$- 1.5780e+1$	
				$- 8.1524e-1$	$- 3.5656e+0$	
				$- 3.2233e+0$	$- 1.1707e+1$	
				$- 6.0435e+0$	$- 1.9620e+1$	
				$- 3.7407e-1$	$- 6.9359e-1$	
				$- 6.3262e-1$	$- 9.9831e-1$	
				$- 5.8514e-1$	$- 1.0619e+0$	
				$- 1.2050e-1$	$- 2.2291e-1$	
				$- 3.4507e-1$	$- 5.3310e+6$	
				$- 5.9000e+6$	$- 3.7716e+6$	
				$- 4.4221e+3$	$- 2.8682e+4$	
				$- 3.4962e+5$	$- 3.4766e+0$	
				$- 1.4386e+1$	$- 1.5020e+2$	
	10 30	$2.2000e+2$	$- 1.4000e+2$	$\approx 1.4000e+2$	$\approx 1.5190e+2$	$\approx 1.5020e+2$
	+/ - / $\approx$	5/28/3	1/20/15	6/22/8	11/18/7	

MOEA/D-DE, FCS-MOEA/D-DE, CPS-MOEA on UF1-10 and LZ1-9.

Data underlying these tables selective, chosen by authors

Selection and analysis differently!

2. HV-based comparison of IP3 with eight different ML methods, where k NN as baseline for pairwise comparisons. The symbols “+”, “=” or “-” indicate whether the corresponding ML method is statistically better, equivalent, or worse, than k NN. T_{avg} denotes the average of the termination generations for 21 runs of the baseline.

M	t_{FM}	RF	ExTree	SVR	ENet	Ridge	LR	XGB	kNN
2	1036	0.529465-	0.535453-	0.524300-	0.541379-	0.581979+	0.531304-	0.507512-	0.557609
2	1157	0.661312	0.658540	0.659964-	0.659488-	0.661387-	0.664115+	0.660775-	0.662832
2	1200	0.681860-	0.681860-	0.681860-	0.681860-	0.681860-	0.681861-	0.681859-	0.681860
2	1263	0.348794-	0.348794-	0.348794-	0.348794-	0.348794-	0.348794-	0.348794-	0.348794
2	1003	1.068499+	1.068395+	1.068444+	1.068469+	1.068446+	1.068464+	1.068577+	1.068348
2	1755	0.681860-	0.681860-	0.681860-	0.681860-	0.681860-	0.681860-	0.681860-	0.681860
2	1812	0.332970-	0.330529-	0.327835-	0.328804-	0.336573-	0.328397-	0.336123-	0.33394
$(=/-) \rightarrow$		0/5/2	0/6/1	0/4/3	0/6/1	1/5/1	1/5/1	1/4/2	of 7 probs.
5	1402	2.486749+	2.486892+	2.486839-	2.486717-	2.486795-	2.486833-	2.486964-	2.486885
10	2023	17.757703-	17.757702-	17.757693-	17.757704-	17.757690-	17.757684-	17.757683-	17.757696
5	866	2.171945-	2.172083+	2.171872-	2.172031+	2.172102+	2.172082+	2.172134+	2.171782
10	757	1.6644176+	1.664258+	1.664655+	1.664885+	1.664505+	1.664448+	1.664356+	1.664525
5	1198	2.083931-	2.079724-	2.049543-	2.057957-	2.033053-	2.055357-	2.099911-	2.09651
10	1851	17.659759-	17.660024-	17.658199-	17.658355-	17.656276-	17.659139-	17.663084-	17.660877
5	915	2.173070-	2.173172-	2.173212-	2.173206-	2.173125-	2.173185-	2.173211-	2.173235
10	800	17.676593-	17.67686-	17.67699-	17.676891-	17.676909-	17.676767-	17.677003-	17.676834
5	853	0.059759+	0.059340-	0.059473-	0.059903-	0.059967-	0.059876-	0.05956-	0.060380
10	1059	0.002074+	0.002155+	0.001959-	0.001957-	0.002004-	0.002149+	0.002159+	0.002000
5	361	0.999336-	0.997776-	0.999999-	1.000740+	1.002444+	0.998881-	0.999796-	0.995748
10	484	7.877194-	7.865008-	7.850738-	7.838514-	7.827113-	7.833383-	7.827973-	7.830099
5	2416	2.488320-	2.488320-	2.488320-	2.488320-	2.488320-	2.488320-	2.488320-	2.488320
10	3658	17.757727-	17.757727-	17.757727-	17.757727-	17.757727-	17.757727-	17.757727-	17.757727
5	694	2.480958-	2.469060-	2.482613-	2.481608-	2.475904-	2.479125-	2.482338-	2.477745
10	1049	17.529054-	17.534092-	17.541146-	17.497915-	17.565631-	17.535149-	17.542442-	17.557395
5	1113	2.487942-	2.487944-	2.487942-	2.487946-	2.487946-	2.487943-	2.487947-	2.487944
10	1197	17.757727-	17.757727-	17.757727-	17.757727-	17.757727-	17.757727-	17.757727-	17.757727
5	1074	0.968845-	0.972045-	0.967674-	0.969739-	0.968382-	0.968788-	0.969065-	0.969353
10	734	7.598982-	7.578182-	7.590612-	7.585287-	7.615941-	7.594704-	7.592734-	7.584186
5	1361	0.000551-	0.000477-	0.000426-	0.000450-	0.000410-	0.000470-	0.000449-	0.000495
10	1046	0.000065-	0.000067-	0.000076+	0.000077+	0.000074+	0.000069-	0.000076+	0.000068
5	1820	0.028026-	0.028181-	0.025498-	0.026383-	0.027629-	0.028588-	0.026612-	0.028363
10	725	0.000065-	0.000080-	0.000099-	0.000056-	0.000035-	0.000061-	0.000072-	0.000080
5	1069	0.964888-	0.957266-	0.967990+	0.965795-	0.968337+	0.959917-	0.964241+	0.961056
10	783	6.060485-	5.912519-	5.959022-	6.296578-	5.875247-	5.879183-	6.002479-	5.908347
5	960	2.448849-	2.449593-	2.450035-	2.449182-	2.449345-	2.449839-	2.450457-	2.446909
10	1231	17.561243-	17.571667-	17.567886-	17.551676-	17.550847-	17.558191-	17.549476-	17.557456
5	546	1.782436-	1.784627-	1.785285-	1.782553-	1.783997-	1.78398-	1.785365-	1.784192
10	481	13.118814-	13.118862-	13.130149-	13.165706-	13.122503-	13.077448-	13.243644+	13.041259
5	911	0.246188-	0.246621-	0.237909-	0.242349-	0.236794-	0.238086-	0.241078-	0.236727
10	904	0.237637-	0.236821-	0.230929-	0.192100-	0.225026-	0.229397-	0.235592-	0.226286
$(=/-) \rightarrow$		1/29/2	2/30/0	2/26/4	5/23/4	4/25/3	2/29/1	5/25/2	of 32 probs.

What do we learn from this?

Lots of interesting data underlying these tables
Limited to one perspective, chosen by authors

Look at data collection and performance analysis differently!

Data Underlying Performance Tables

	F1	F2	F3	F4	F5	F6	F7	F8	F9	F10
Mean AE (Average error)	1.10E-10	1.00E-10	0	0	1.10E-10	0	1.16E-10	0	0	5.53E-09
Std AE	3.01E-11	0	0	0	3.01E-11	0	3.74E-11	0	0	3.02E-08
Mean FE (Function Evaluations)	5656.161	10014.61	22959.06	30293.48	33343.71	49143.23	9943.355	34803.45	29948.03	261120.9
Std FE	1148.396	1974.198	5500.869	6148.313	2769.432	30825.26	1987.059	6299.324	8042.612	154161.8
SR (Success Rate)	1	1	1	1	1	1	1	1	1	0.967742

What does this information tell us?

Function value at end of the run -> mean + std imply aggregation over multiple runs

Function Evaluations to reach specific target value

Two perspectives on the same data

Both have a key parameter: Budget at which the run ends / target to reach

What if these are not fixed, but variable?

Show more aspects of the algorithm's performance

Anytime Performance

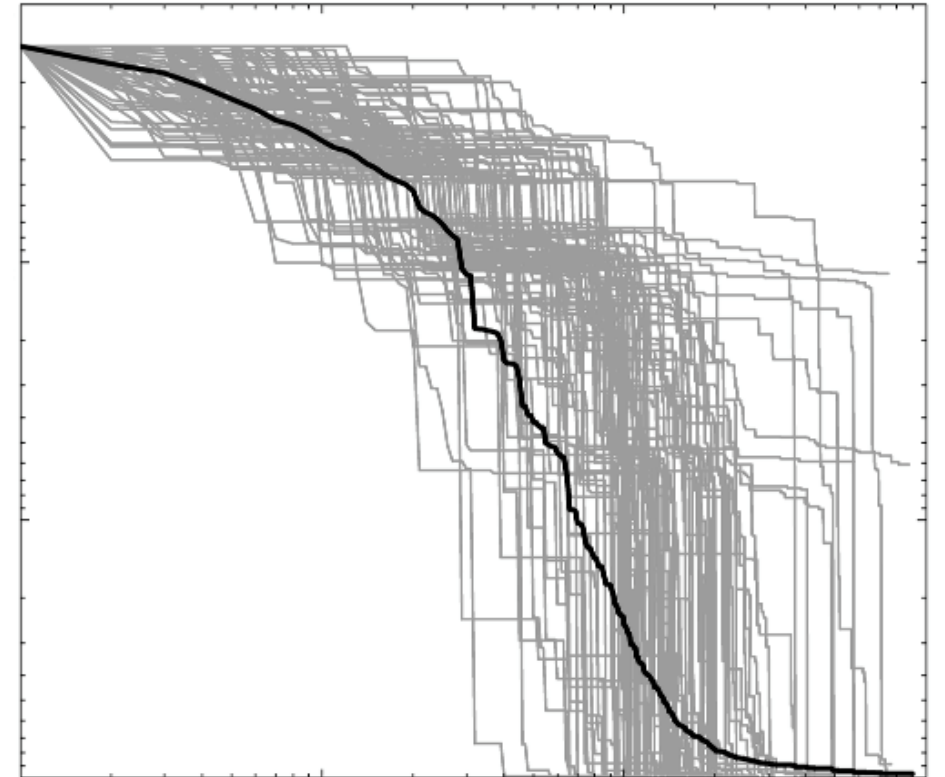
Core idea: we want to know not just where the algorithm ends up, but how it gets there

Performance over time can be very informative

Equivalent for fixed-target perspective: not just one target, but we want to know hitting times for many targets

Capture 'progress' of an algorithm

Third Perspective: Probability!

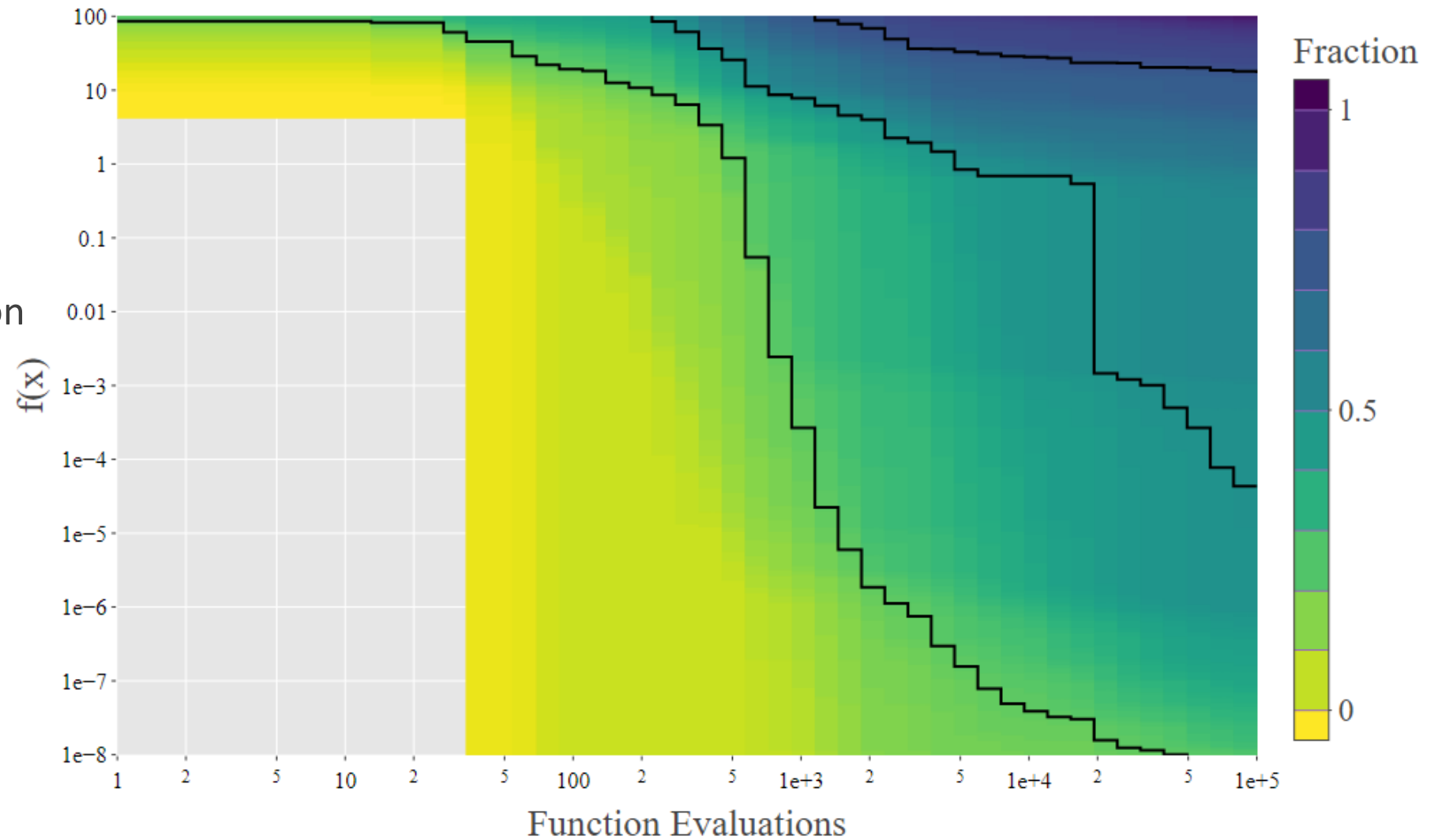


Perspectives on Performance: EAF

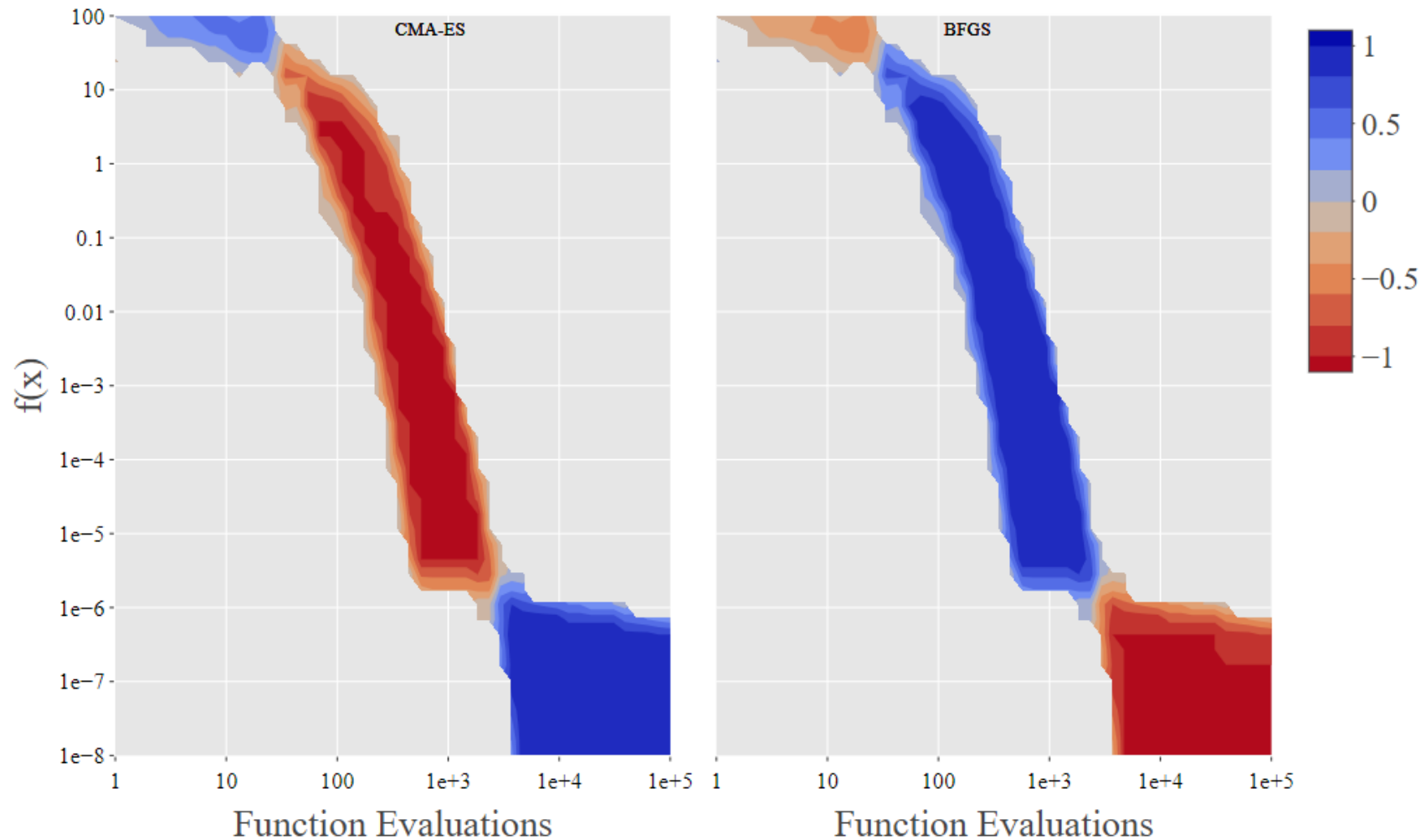
For each $(B, f(x))$:

- What is the probability we reach this $f(x)$ in at most B evaluations?
- Links to pareto-dominance questions from MOO

Empirical Attainment Function



Using Empirical Attainment



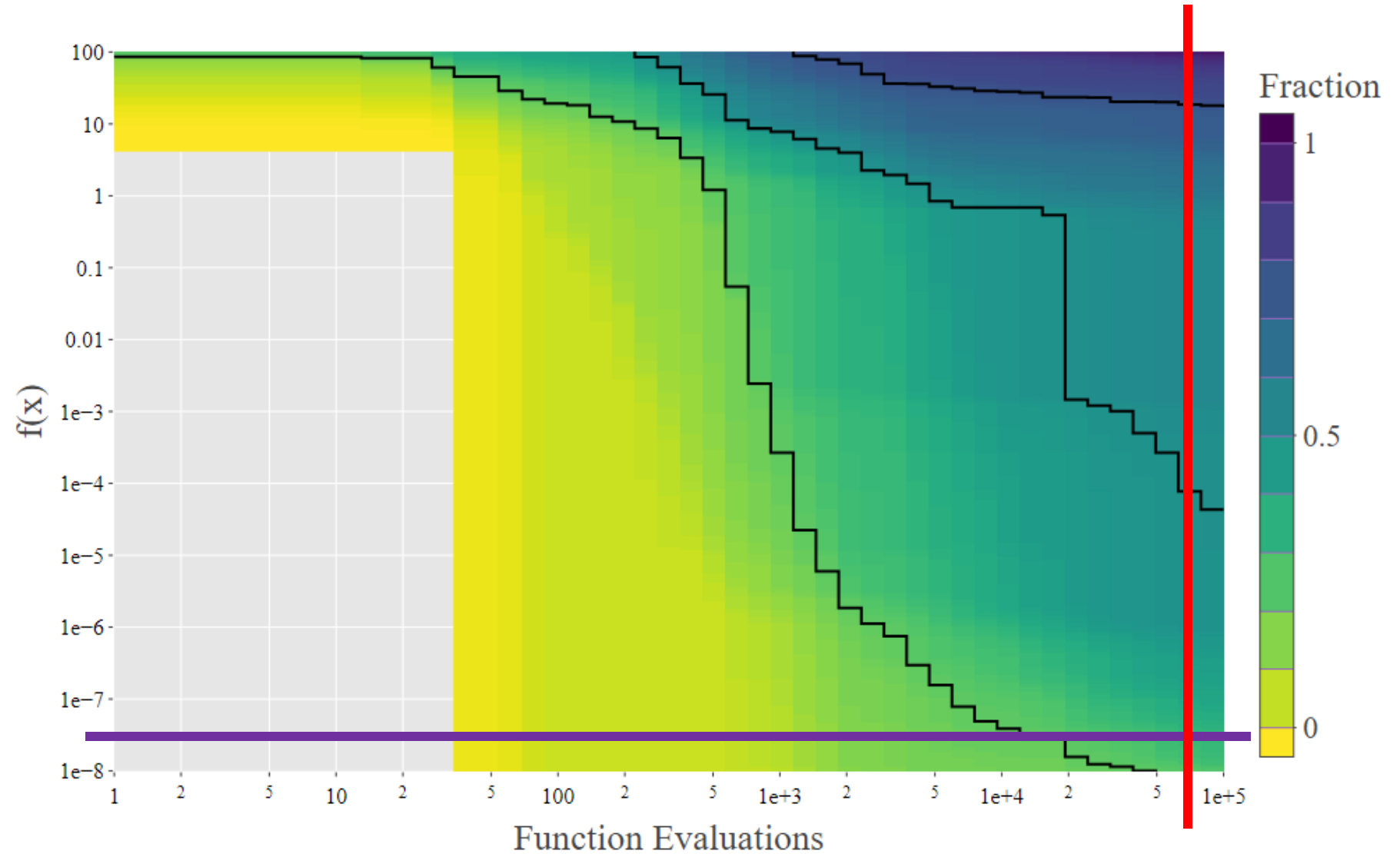
Using Empirical Attainment

Different aggregations of EAF result in common performance measures

Slice in target-space:
Hitting time distribution

Slice in budget-space:
 $f(x)$ distribution

Can even get aggregation in probability-space!



Levels of Aggregation

Looking at per-function, over-time plots can be very informative

But aggregation might be necessary

EAFs can be easily aggregated over multiple functions

- If care is taken to ensure equivalent target-ranges (and budgets)

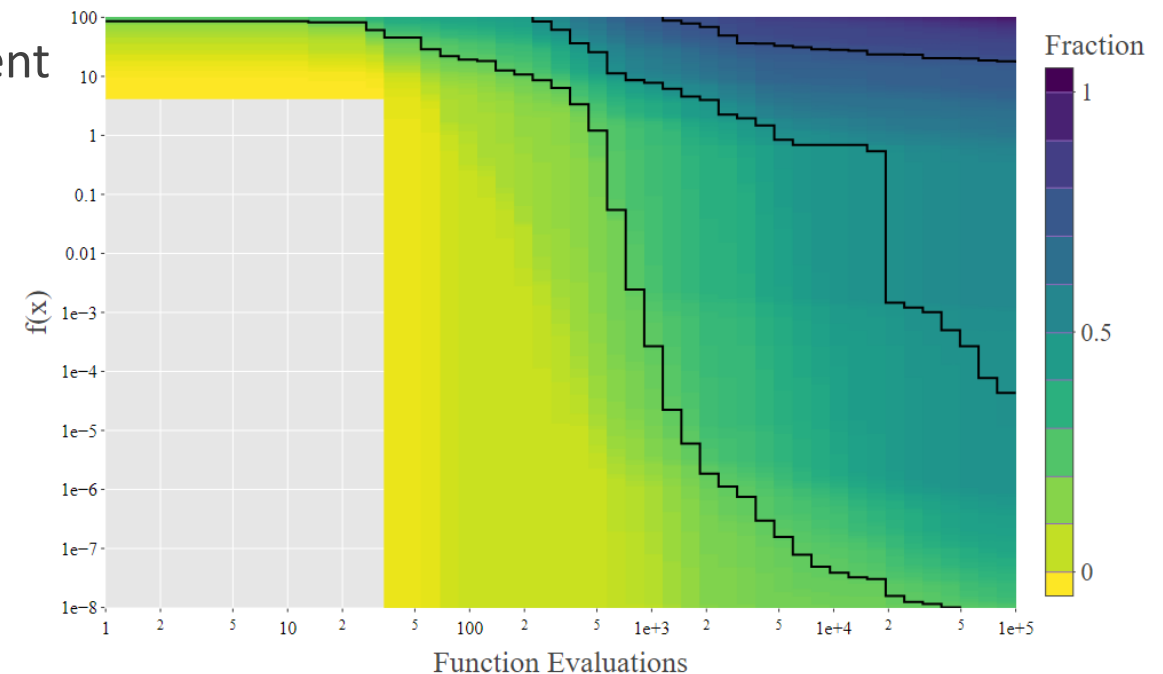
Instead of slices of EAF, can also take (partial) integrals

Partial integral w.r.t budget results in ECDF-equivalent

Double integral results in single measure for **anytime performance**, equivalent to area over convergence curve



Care should be taken when budget explicitly part of algorithm design, e.g. single-shot optimization



Basic Benchmarking Checklist



What is the goal of this benchmarking study?



What algorithm(s) should be included?



What problem(s) do we use?



How many repetitions to use?



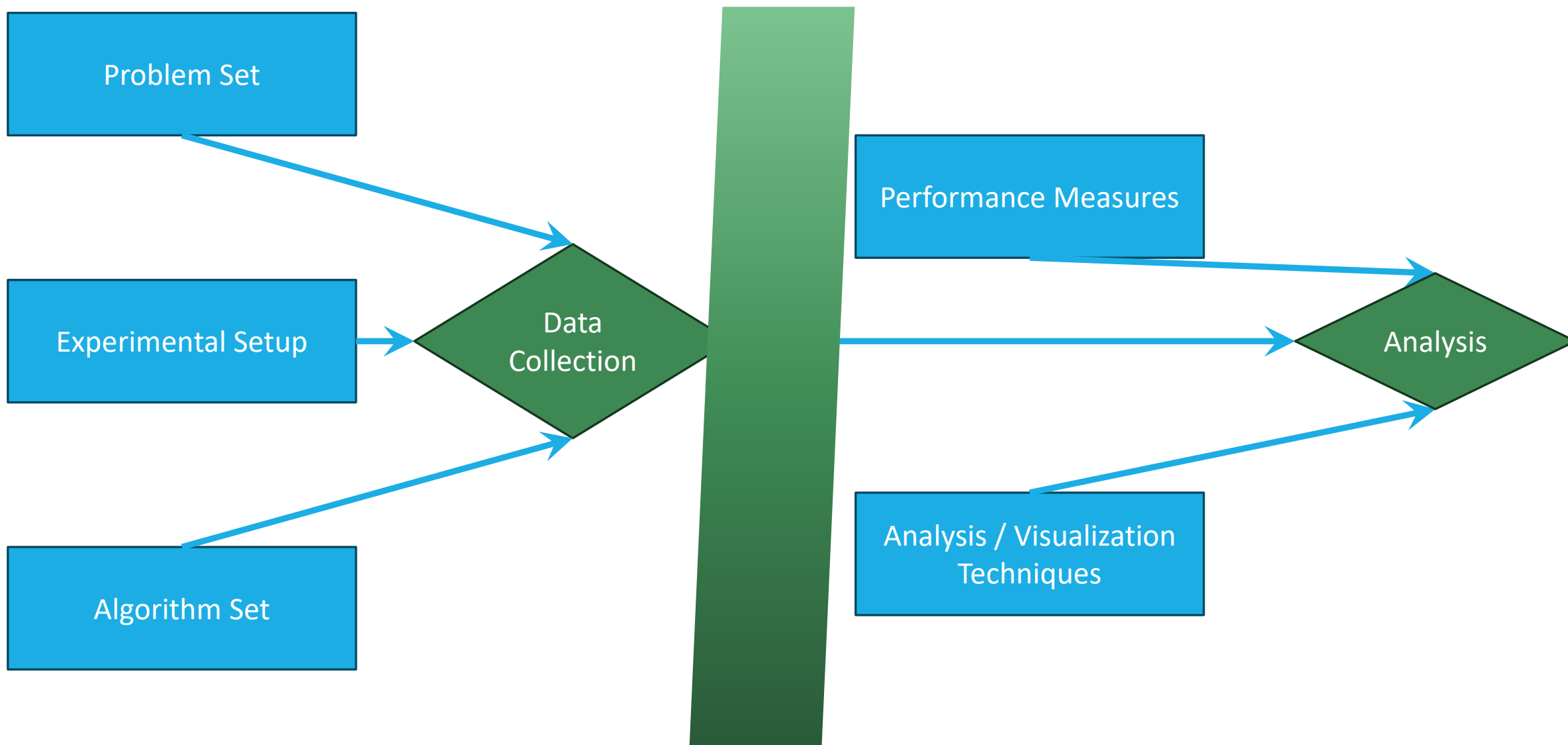
What measure(s) are we interested in?



How to evaluate / visualize the results?



Can we ensure results are reusable / reproducible?



Generating data

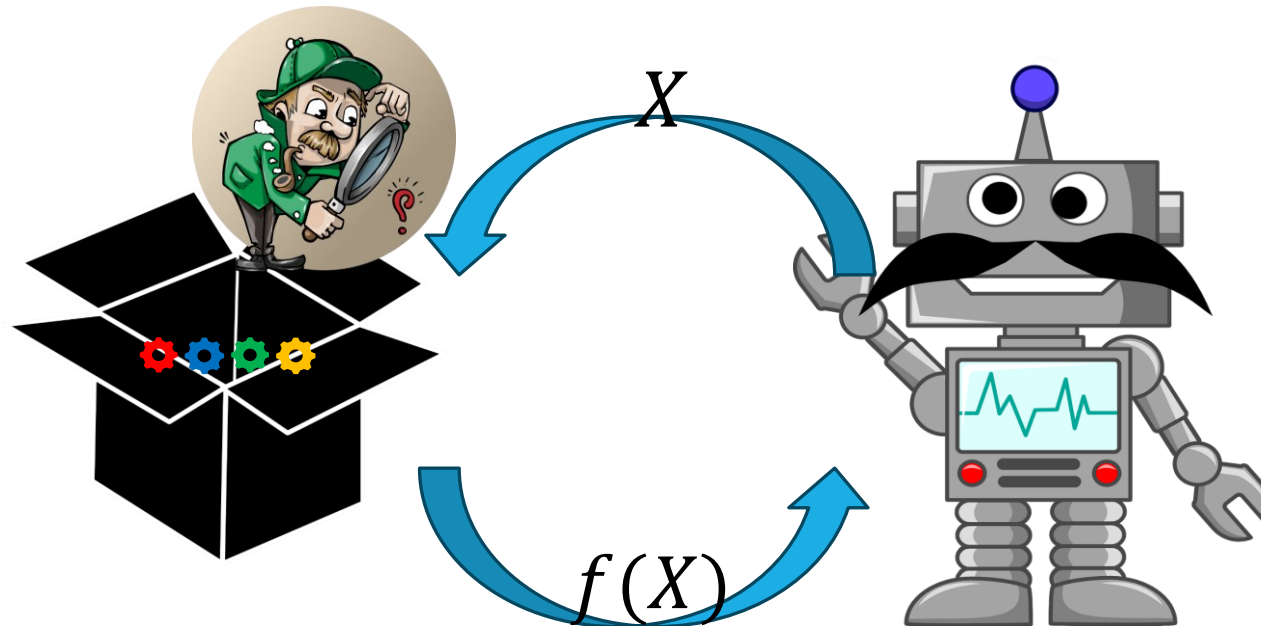
Core benchmarking step is running the algorithm(s) on the problem(s)

Here, we decide what data to collect

Our philosophy: logging from the problem perspective

Create some separation between performance data (collection) and analysis

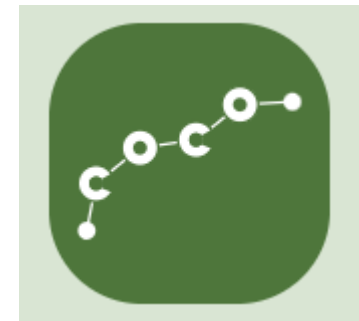
Can also log algorithm-specific properties, but then these should be provided to the logger



The role of software (in doing experiments)

Software should simplify robust benchmarking practices

- Logging: standardized ways of collecting and storing data
- Accessing problems: reduce effort of creating interoperability between problem and algorithms
- Accessing algorithms: simplify using and configuring the optimization algorithms



Basic Benchmarking Checklist



What is the goal of this benchmarking study?



What algorithm(s) should be included?



What problem(s) do we use?



How many repetitions to use?



What measure(s) are we interested in?



How to evaluate / visualize the results?



Can we ensure results are reusable / reproducible?

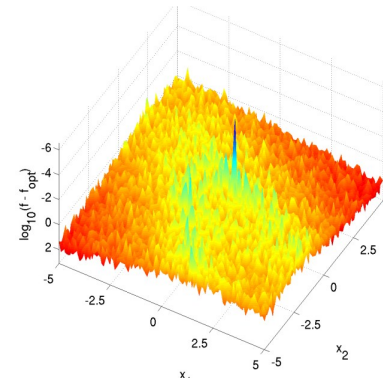
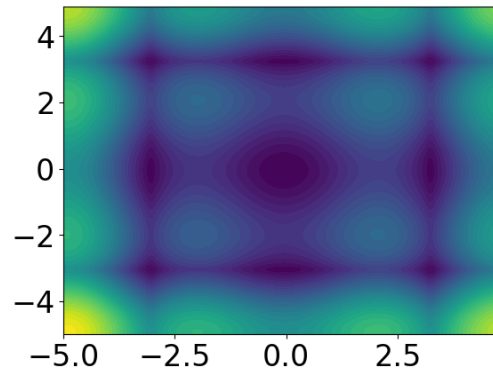
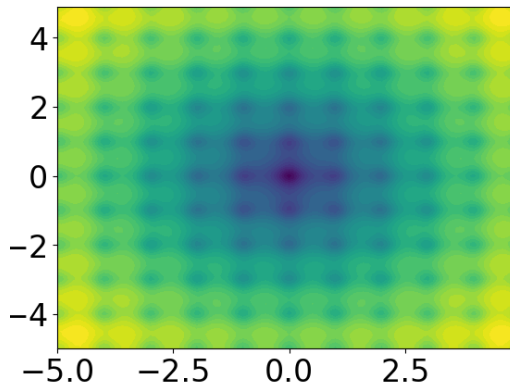
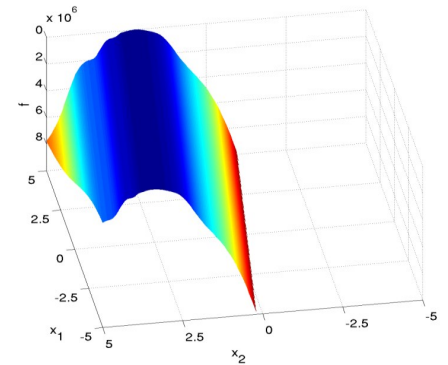
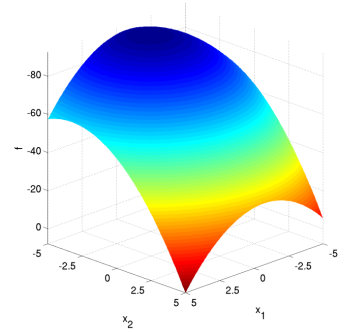
Problem Selection

Set of problems used determines the types of conclusions which can be drawn

Key criteria often aimed for:

- Diversity
- Representativeness

What problems are available? Open question to create accessible collection, COST-action ROAR-NET WG6 would be happy with any input you have on this!



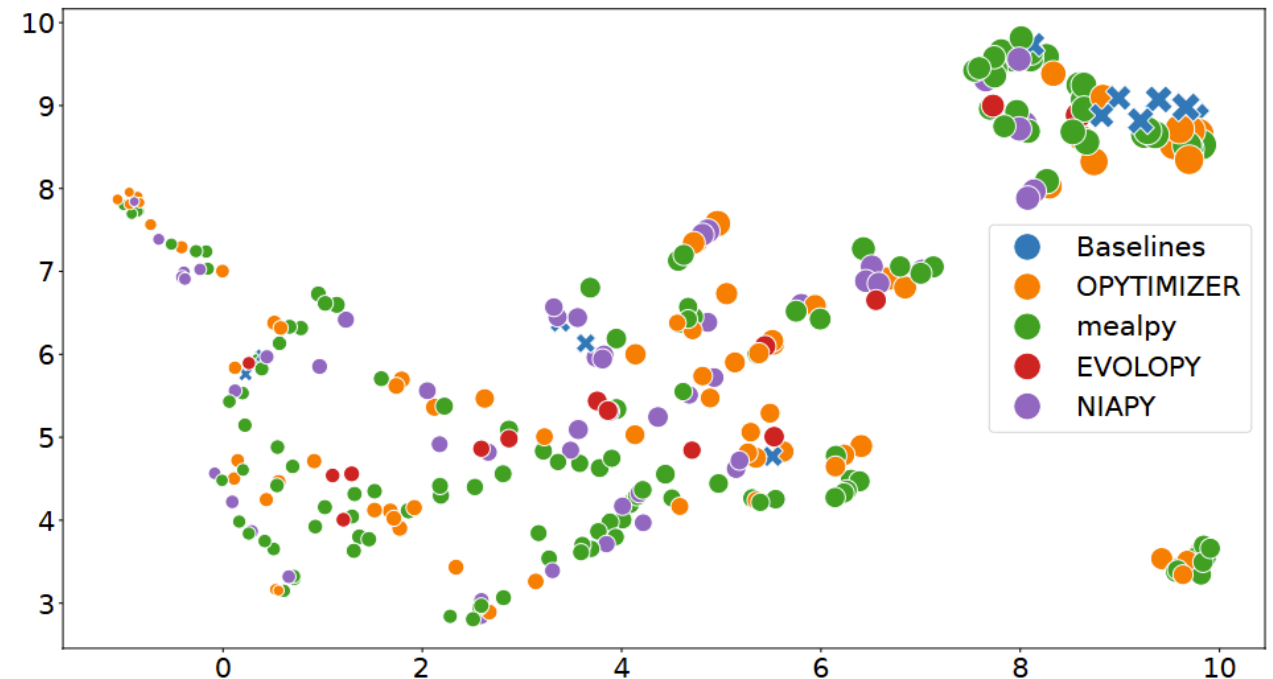
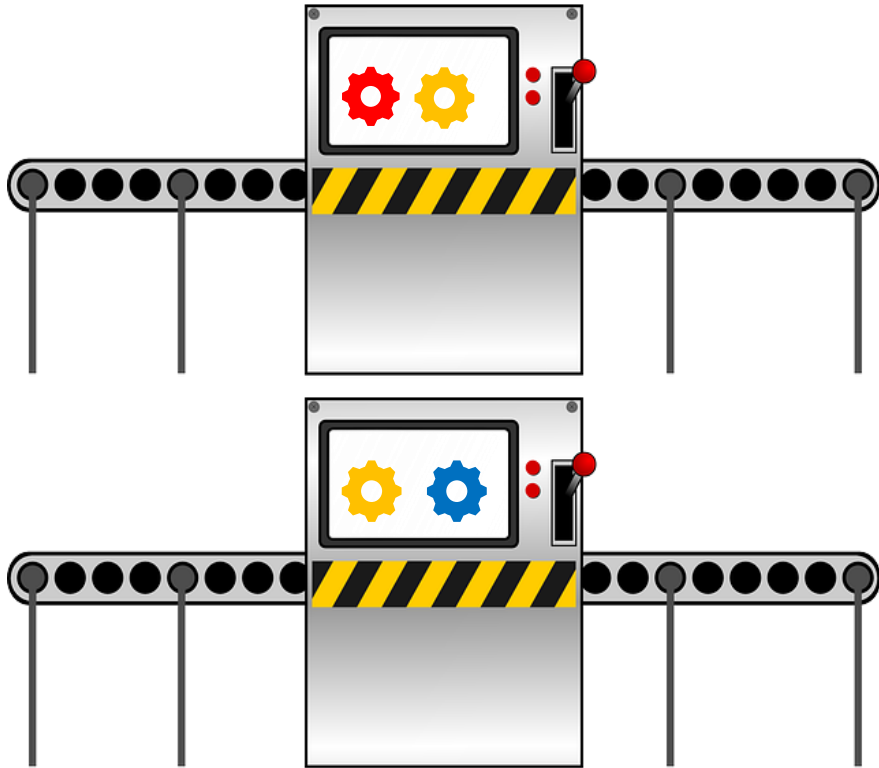
ROAR-NET

Algorithm Selection

For most comparative analyses, the choice of baselines greatly impacts conclusions

Lots of algorithms available

Should contain relevant algorithms for the problem domain



Algorithm Selection: Parameterization

Most algorithms have a set of (hyper)parameters

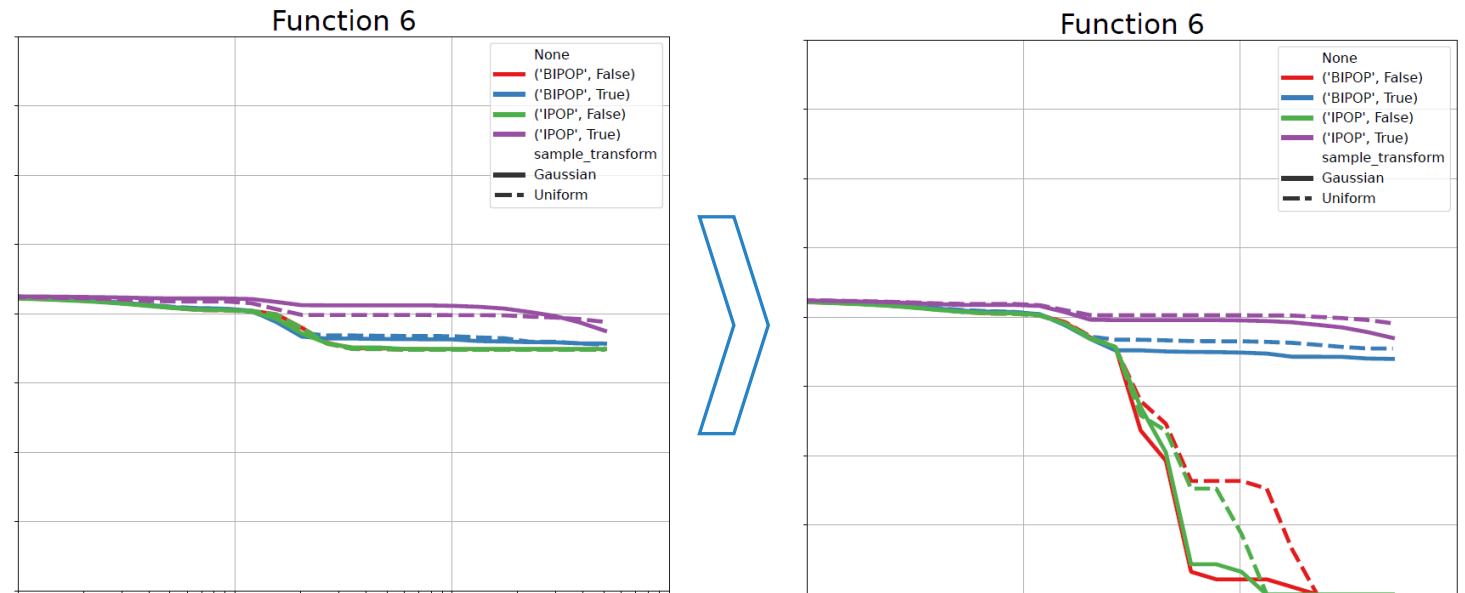
Can have a big impact on performance

- Big difference between PSO with 1 particle vs 30 particles
- Initialization methods?
- Stopping / restart criteria. Example: Same algorithm portfolio, different convergence tolerances

How to ensure fair settings for comparison

Equal HPO effort for all algorithms?

Algorithms should be fully specified



Experimental setups

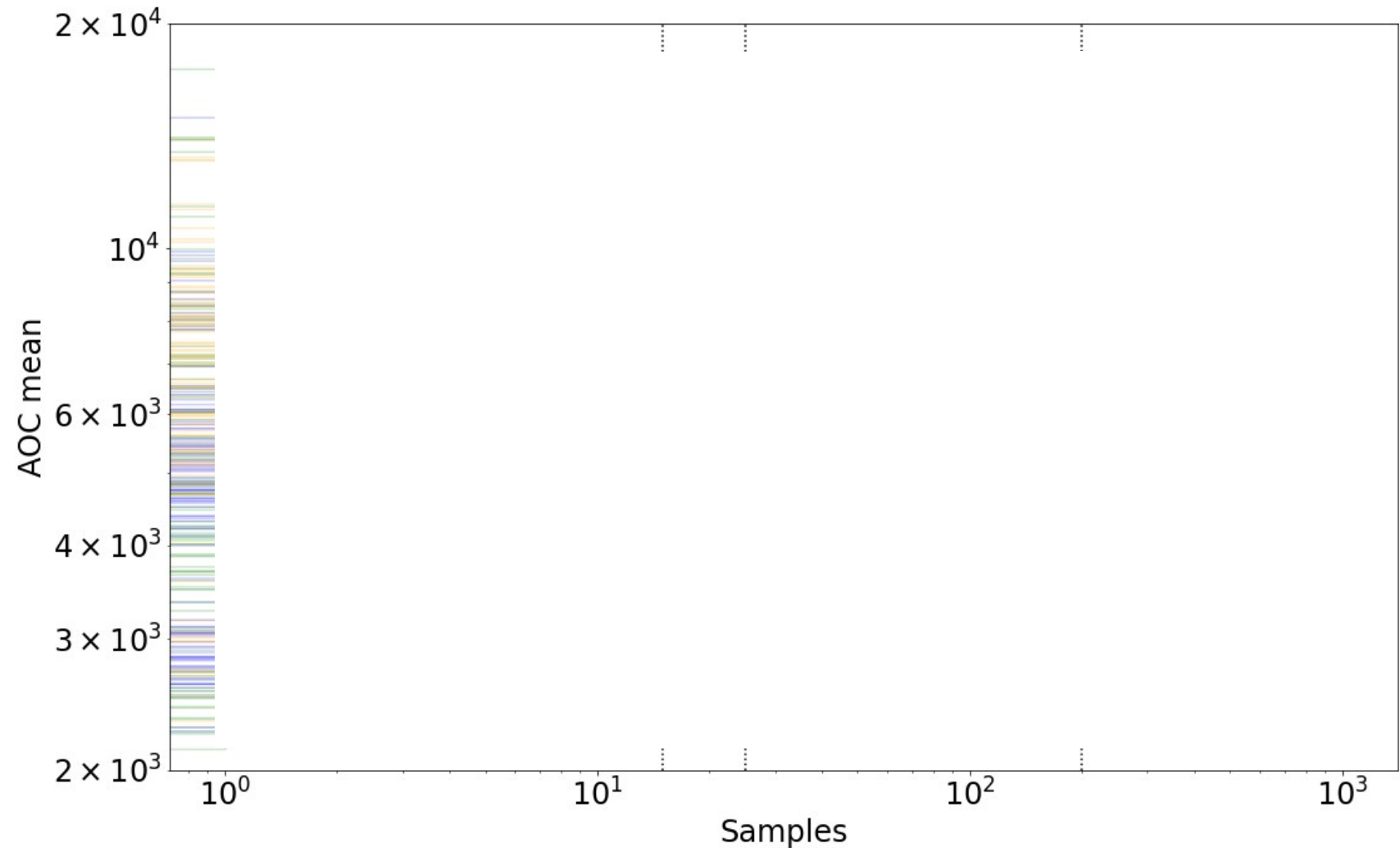
Most decisions made on problem and algorithm level

Budget -> Evaluations

Dependence between runs: same seeds?

Replicability

Number of repetitions



Reproducibility

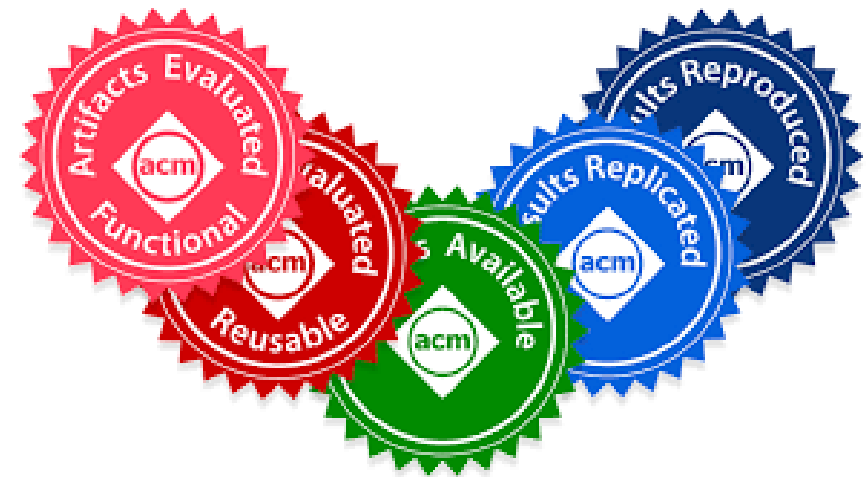
Multiple aspects of reproducibility

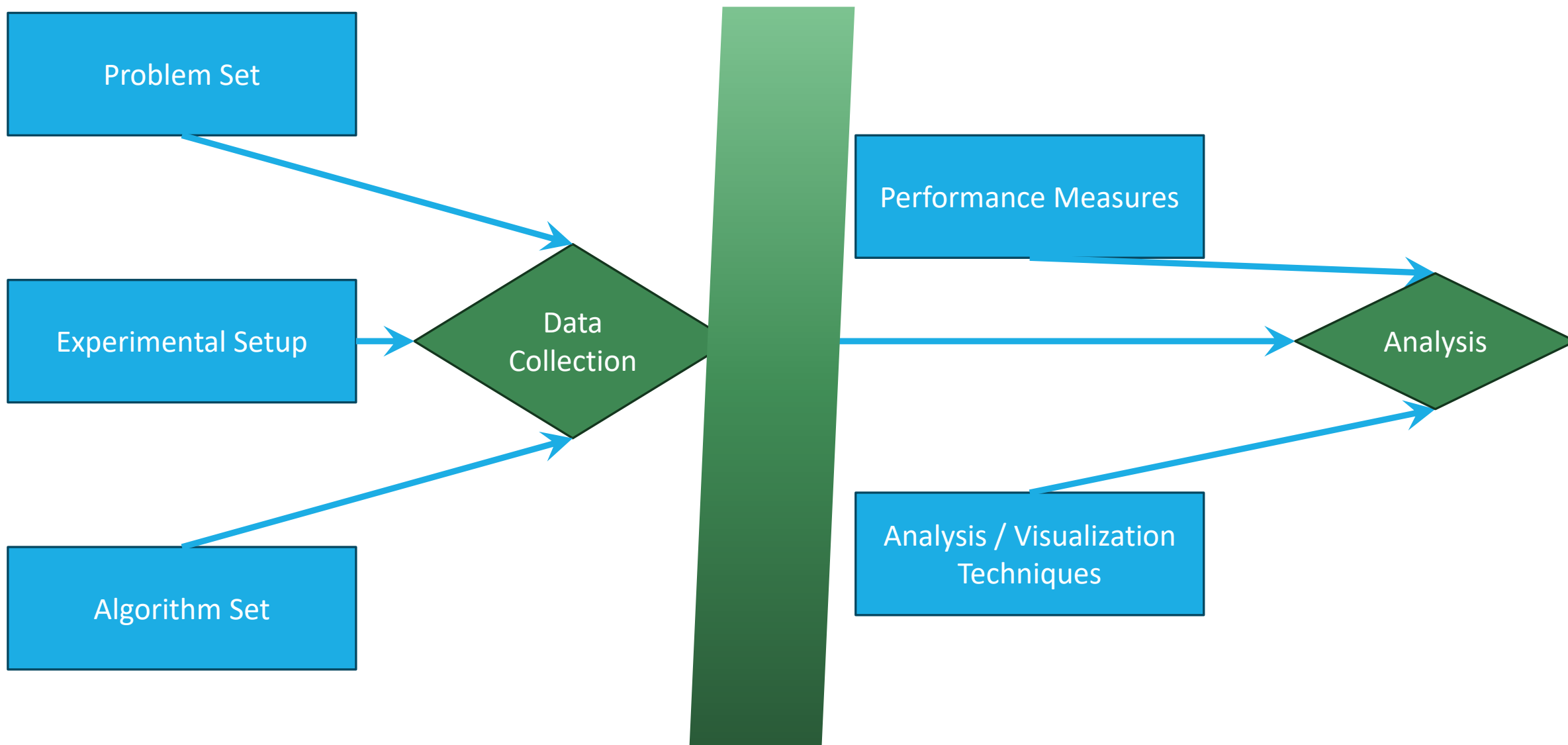
Availability of data and code is the most basic level, should be the minimum standard!

Providing code which is inter-operable with existing frameworks makes it easier to re-use

Without reproducibility, it is harder to trust and thus use provided data in new studies

- E.g. unknown parameter settings
- Data is more useful if the setup to generate it is fully transparent!





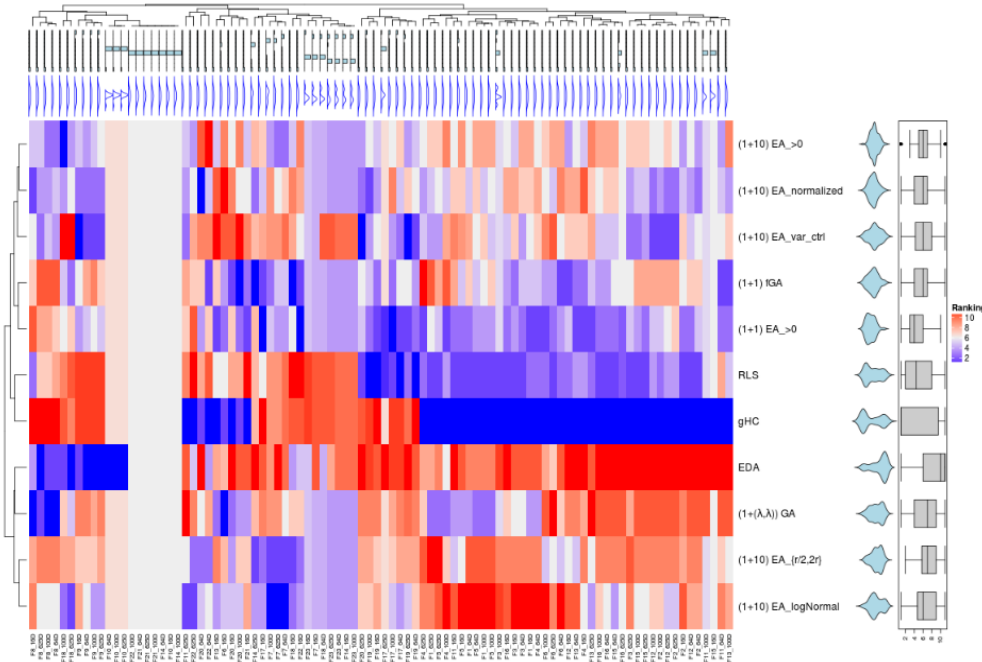
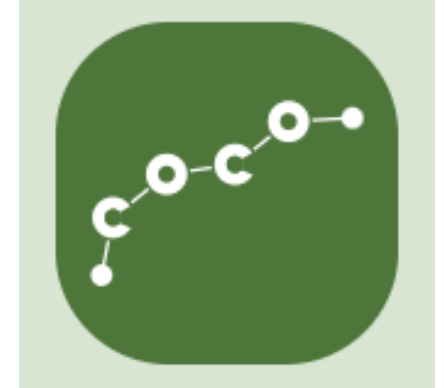
The Role of Software (in analysis)

Process performance logs

Calculate performance measures and aggregations

Statistical analysis methodologies

Visualization methods



Tyranny of the Default

While analysis can be flexible, default settings are critical

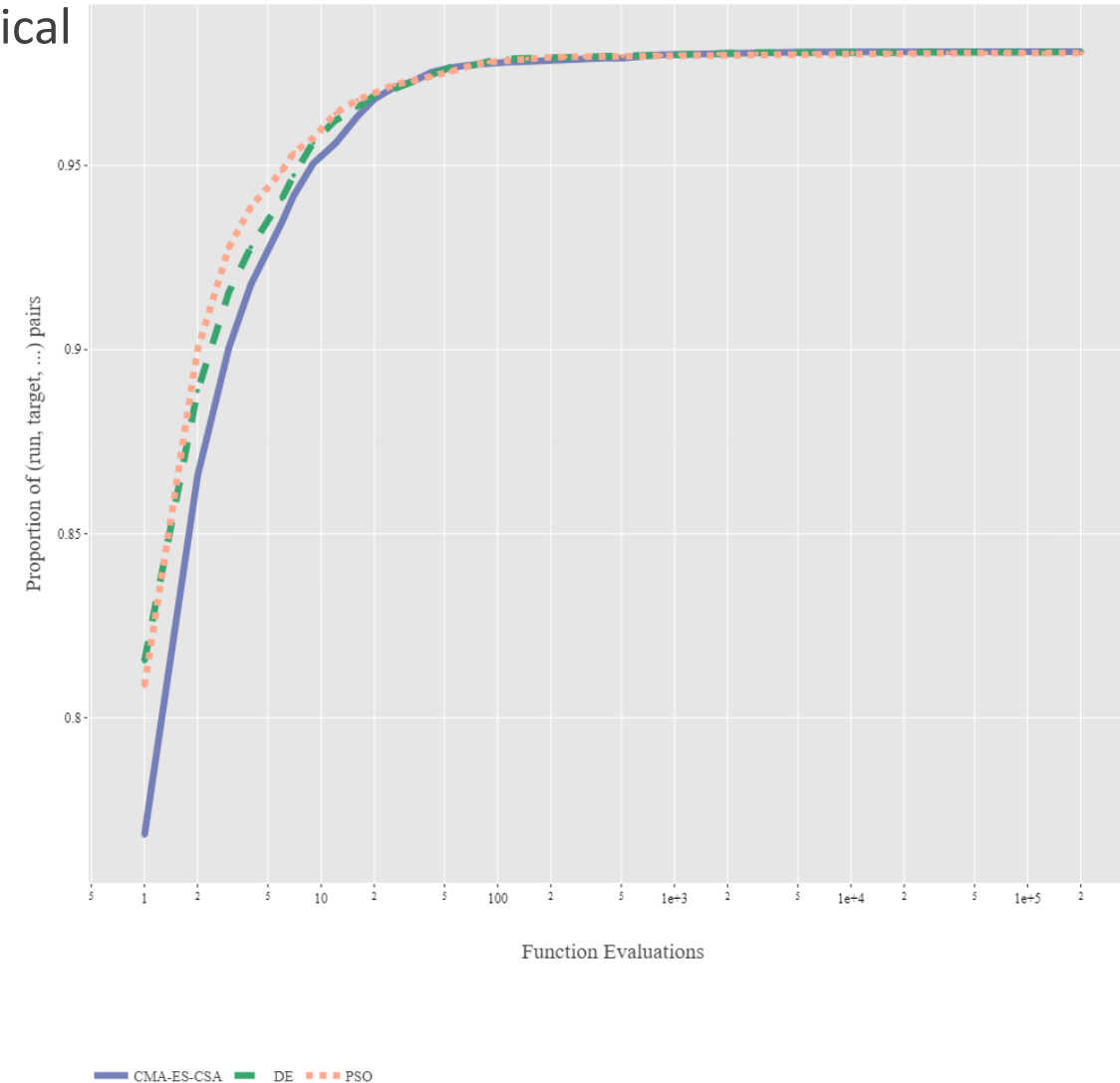
Most users stick with defaults, even when not ideal

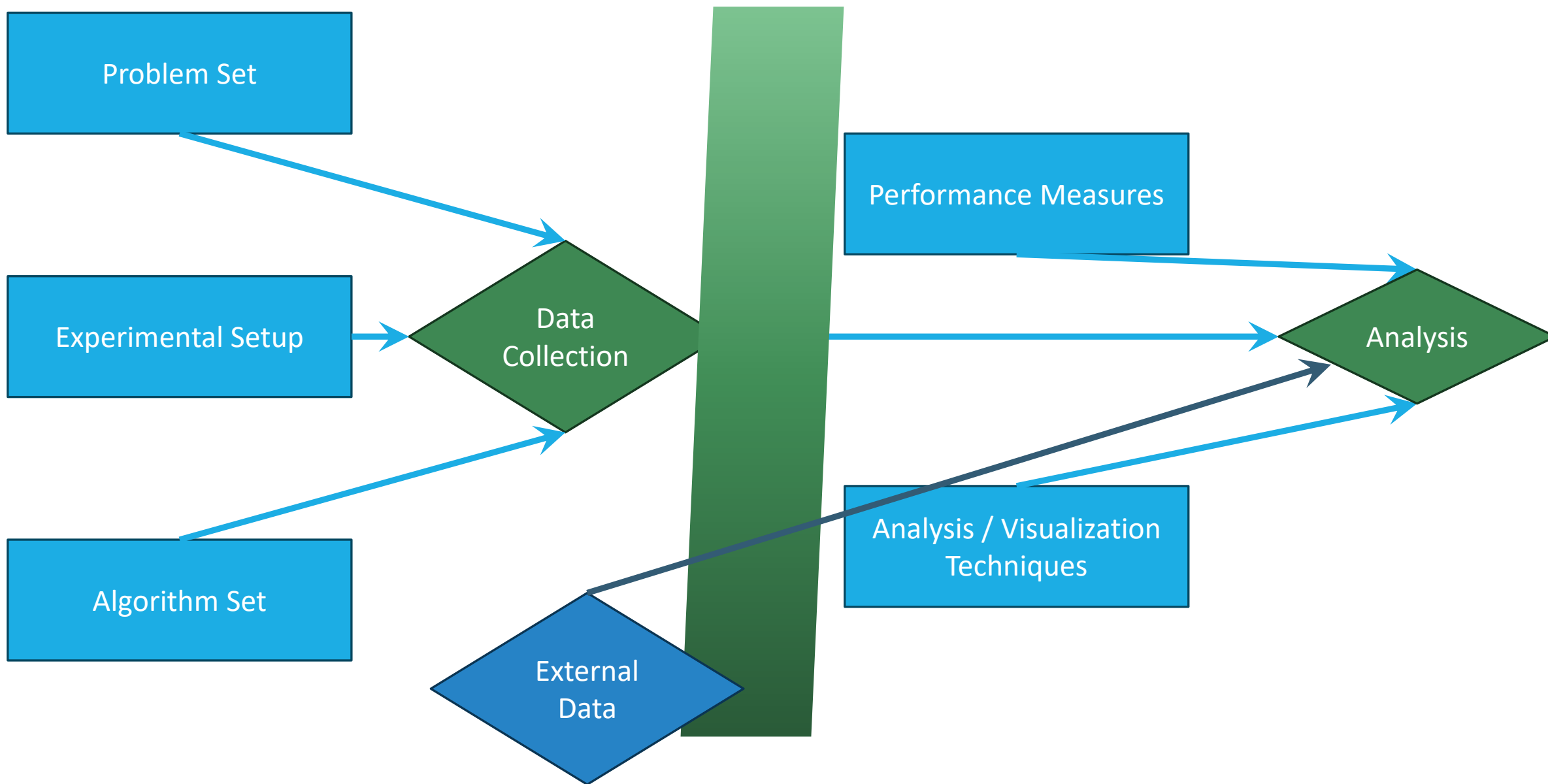
User eXperience (UX) design often overlooked!

Can we converge on reasonable default settings?

Example: targets for ECDF plots

- BBOB defaults to specific set of 51 targets
- How to set for other suites?



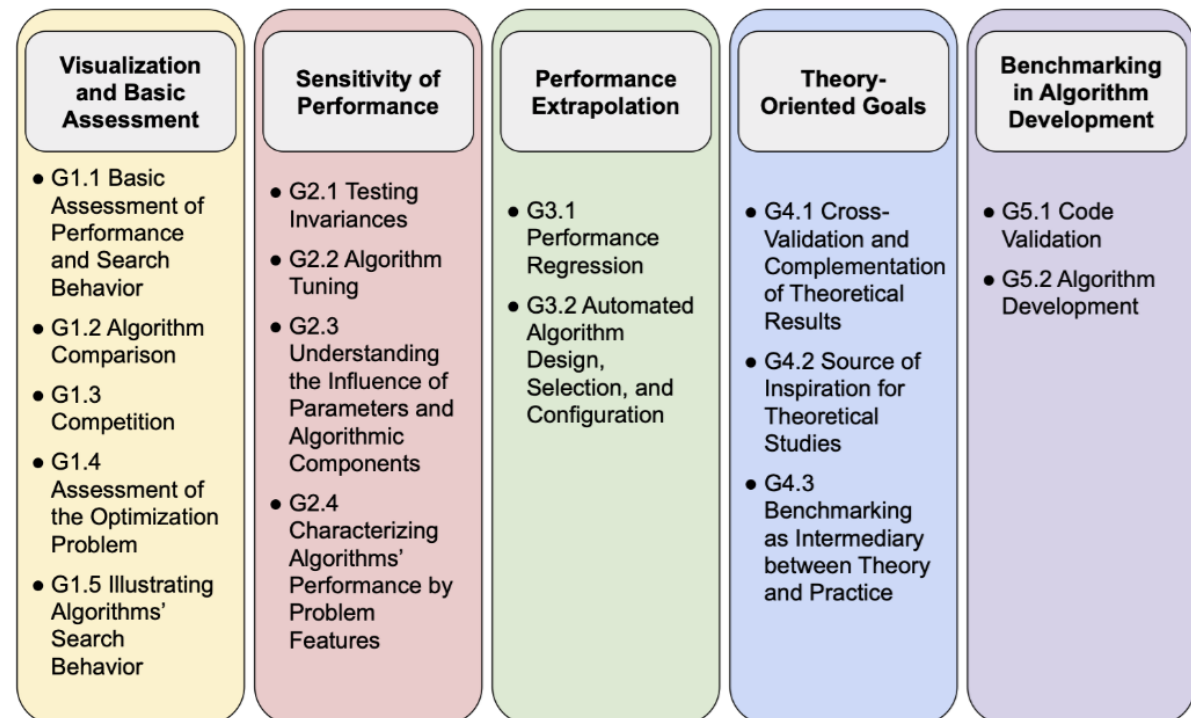


Data sharing and re-use

Separation between experimental setup and analysis decisions make data more useable

For direct comparison, but also for meta-algorithmic studies

Data generated for one goal can be used for others as well!



Meta-algorithmics

With enough performance data, we can work with meta-algorithmic methods

Some examples:

Algorithm A works well on Function 1
Algorithm B works well on Function 2

Algorithm Selection!

Algorithm A works well for low budgets
Algorithm B works well for high budgets

Algorithm Switching!

Parameter value A shows different
strengths than value B

Configuration /
Explainability

No algorithm performs well on problems
3 and 4

New areas for
algorithm design

Problem Features

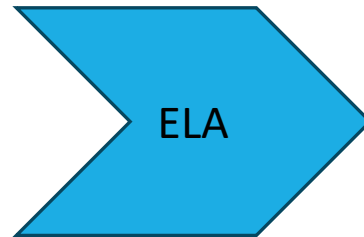
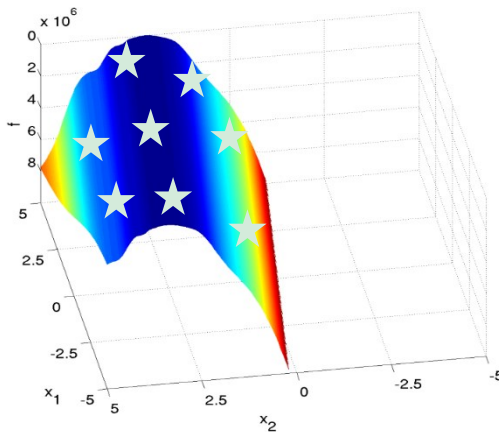
Enrich performance with information about the problem

Just knowing an identifier of a problem is not very useful

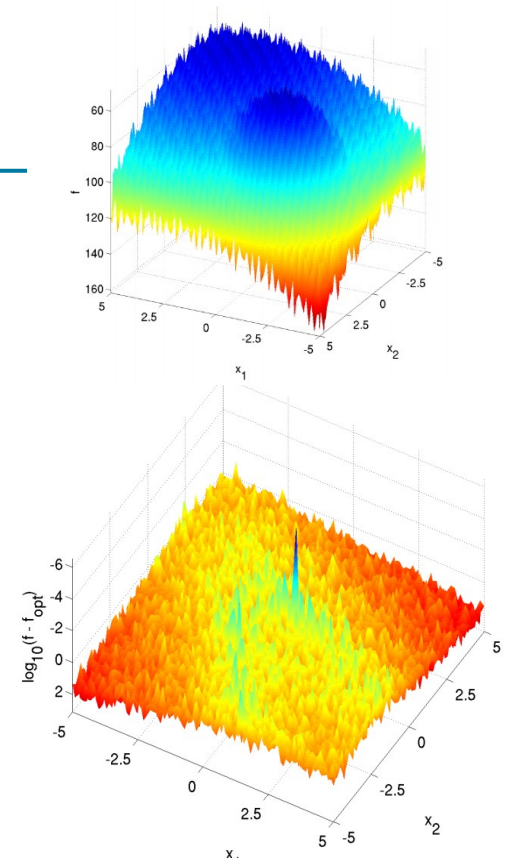
Want to use characteristics of problem

In general, we might not know these for arbitrary problems

Popular method: sampling-based features (ELA)



$$\begin{bmatrix} f_1 \\ f_2 \\ f_3 \\ f_4 \\ \dots \\ f_N \end{bmatrix}$$

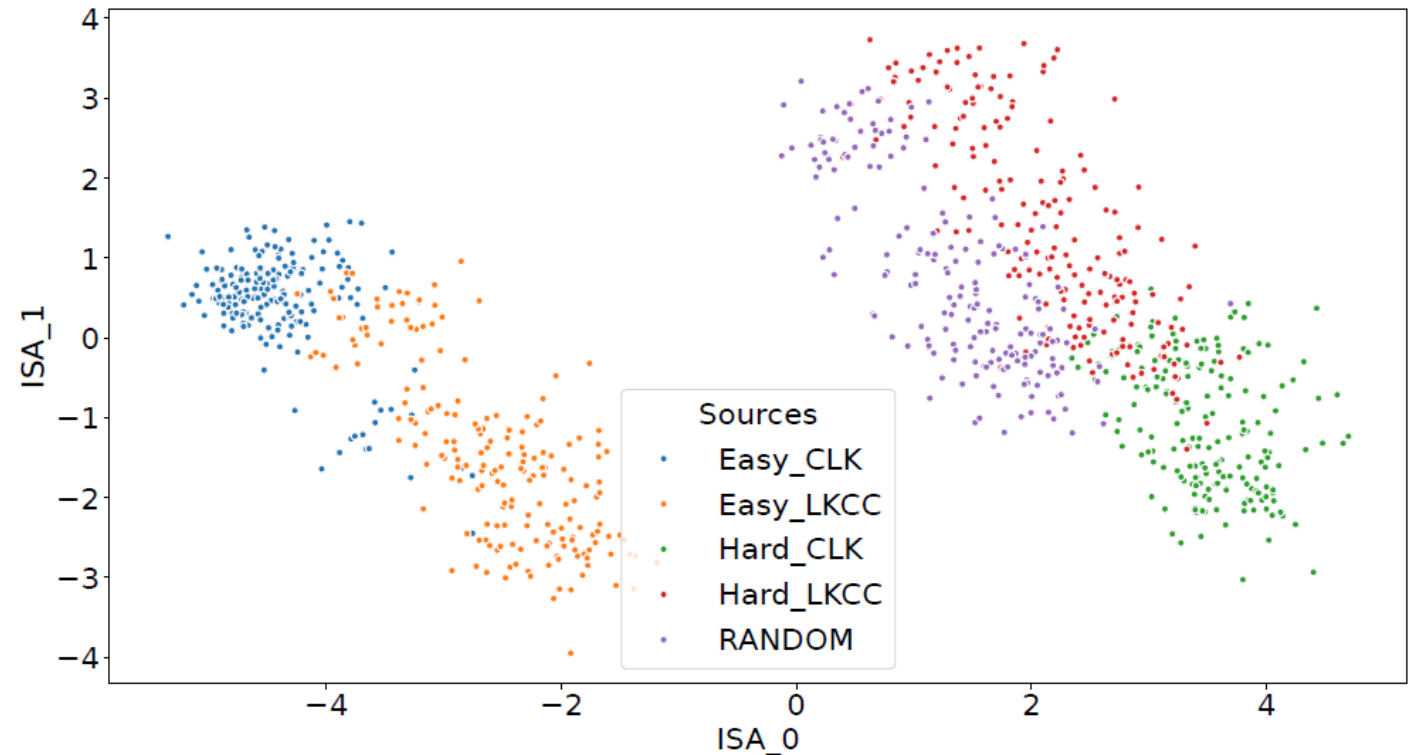


Instance Space Analysis

Linking problem features and algorithm performance

Understanding whether the space of relevant problems is well-covered (from the perspective of a given set of algorithms)

Visualizations of problem space

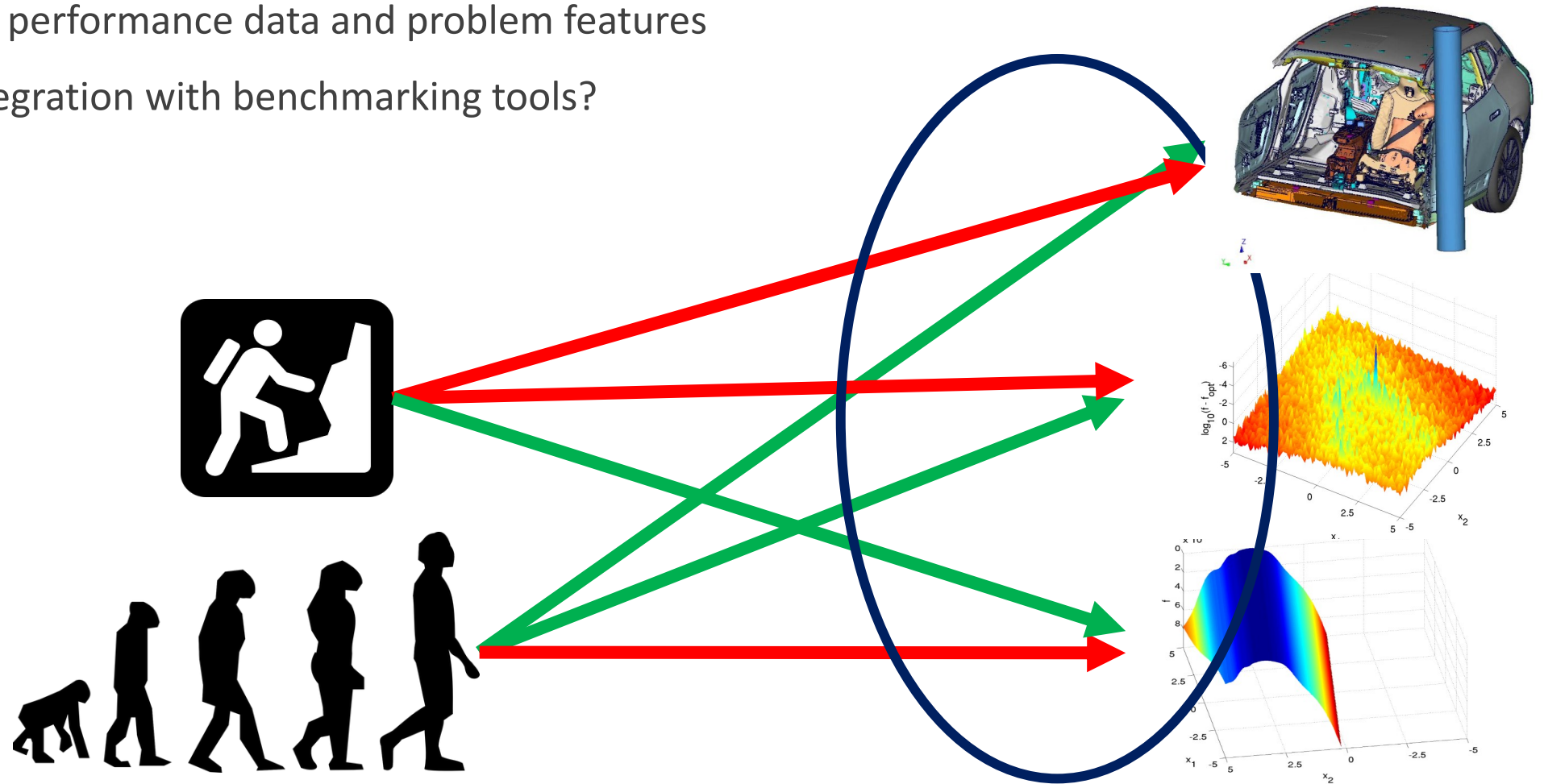


Algorithm Selection

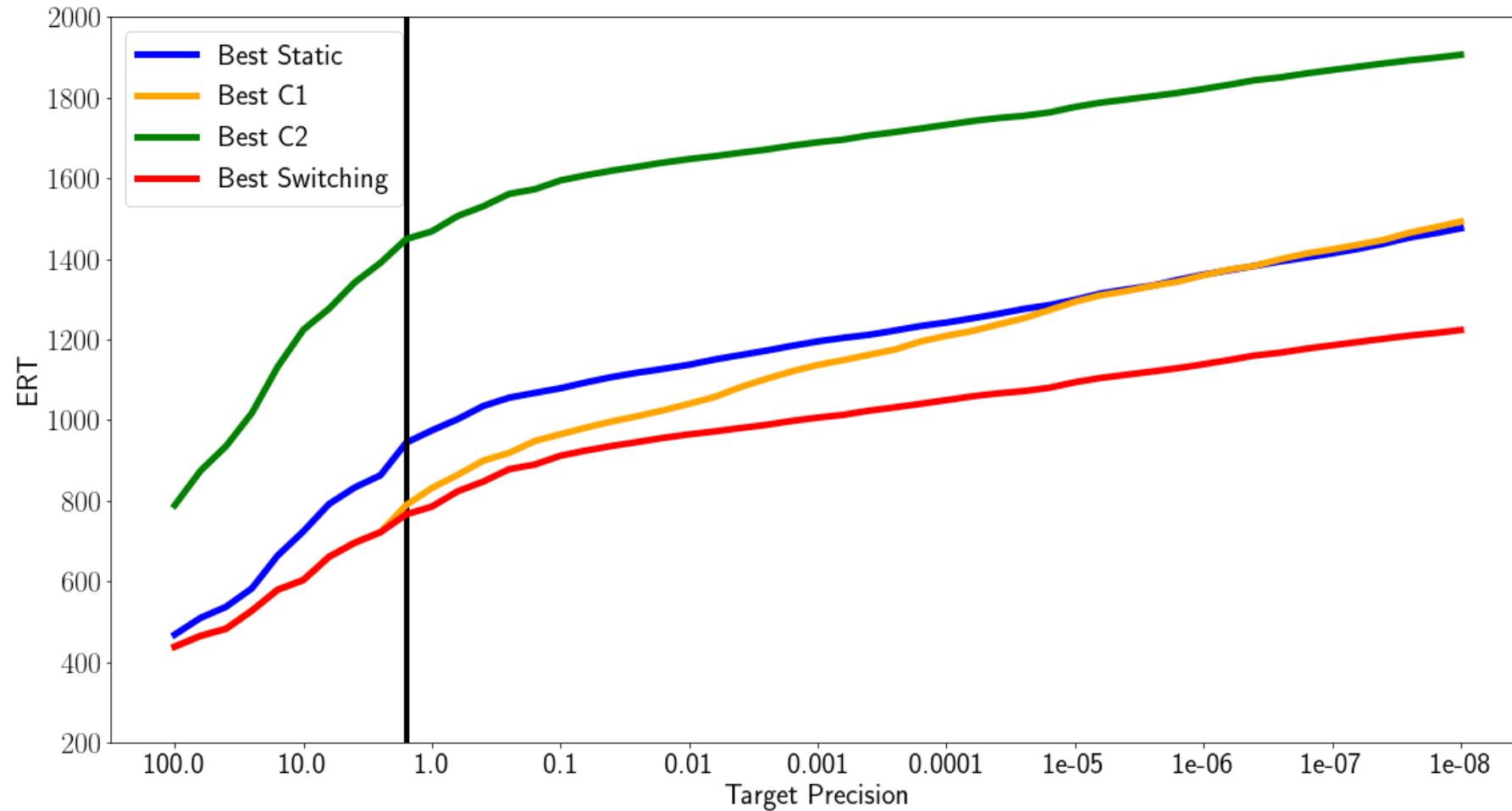
Efforts to collect suitable data sets, e.g. ASlib

Combination of performance data and problem features

More direct integration with benchmarking tools?



Dynamic Algorithm Selection

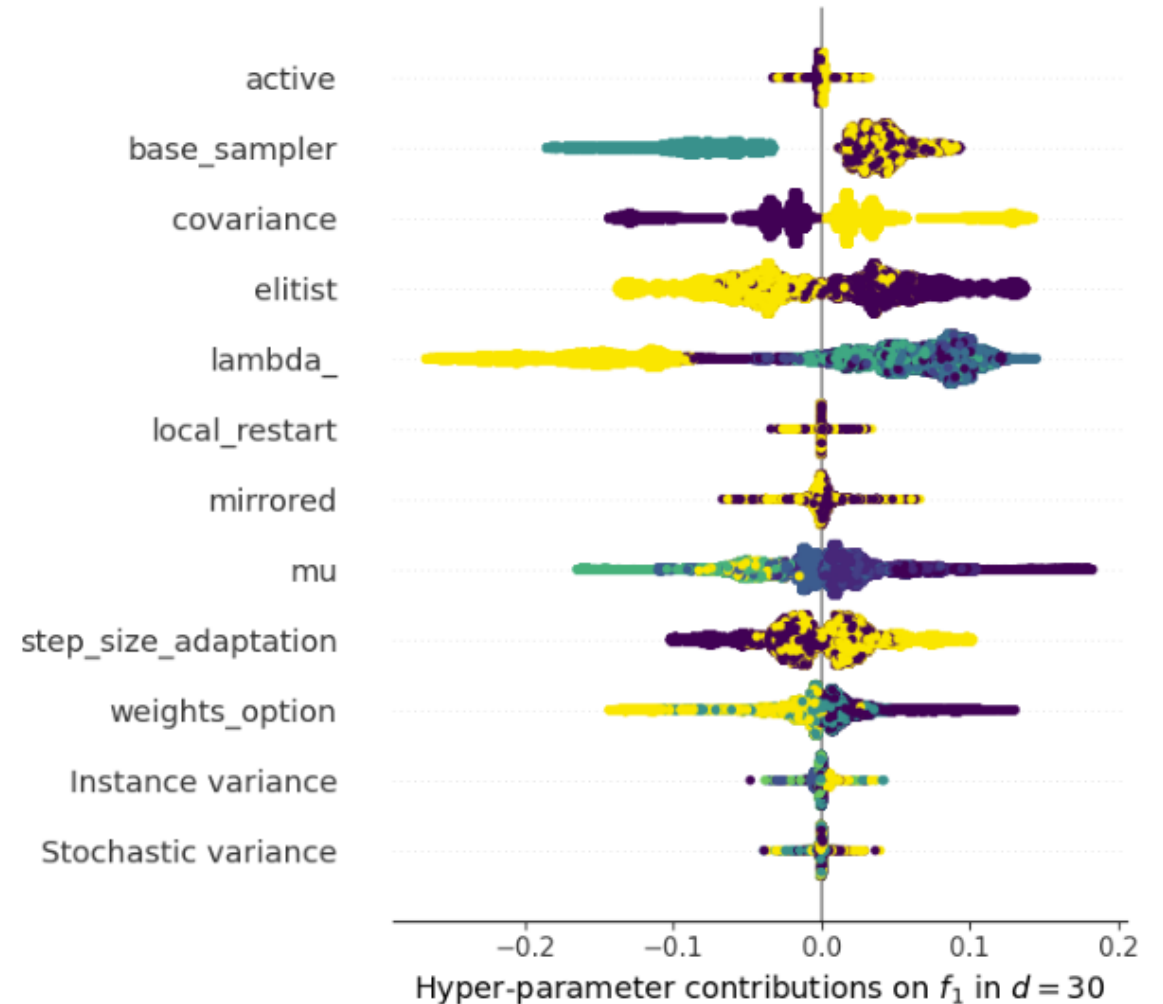
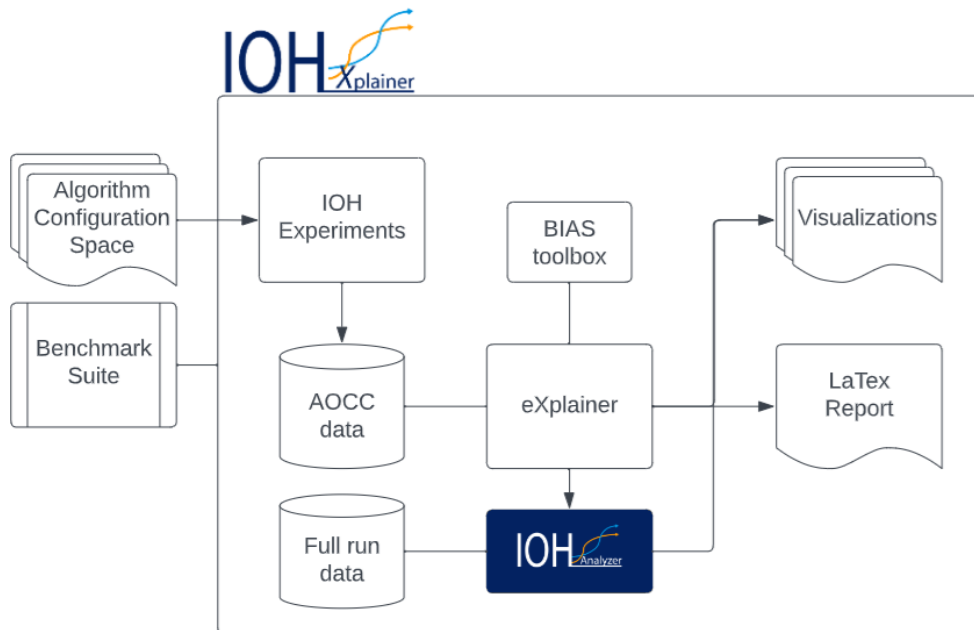


Integrating Explainability

With enough performance data, we can integrate other ML methods as well

Different ways of looking at explainability

Example: analysis of parameter importance in modular algorithms



Sharing Negative Results / Data

Negative results often get ignored

Still contain useful information

Not published / linked to paper -> can't be re-used

Would benefit from repository where any type of data can be freely uploaded



Towards OpenOpt

Central goal: simplify sharing of data

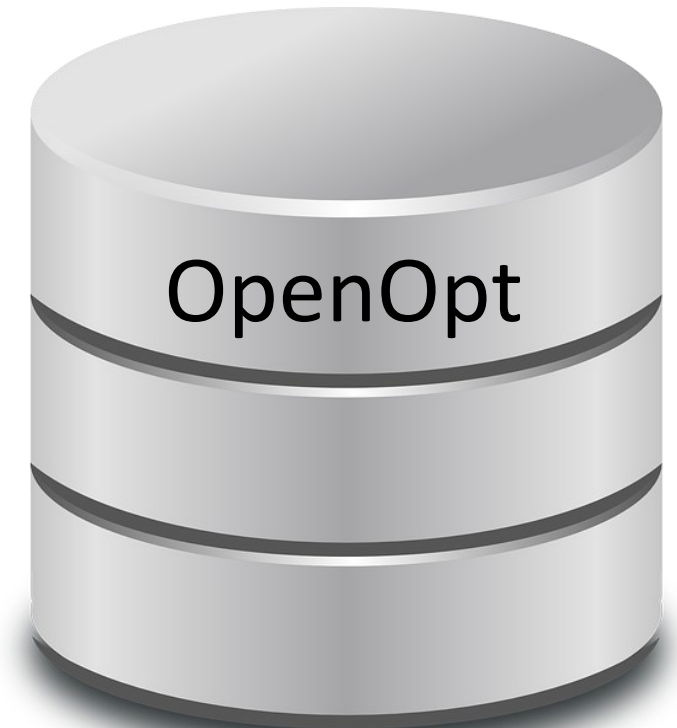
- Primarily performance
- Related meta-data
- Information on algorithms, problems

Central repository, simple interface to contribute data

Standardized data formats (interoperable with existing tools)

Queriable: find and access relevant data

Open access, not dependent on any single benchmarking tool, but interoperable



Multi-Objective Anytime Performance

In MOO, we deal with sets of solutions rather than a single point

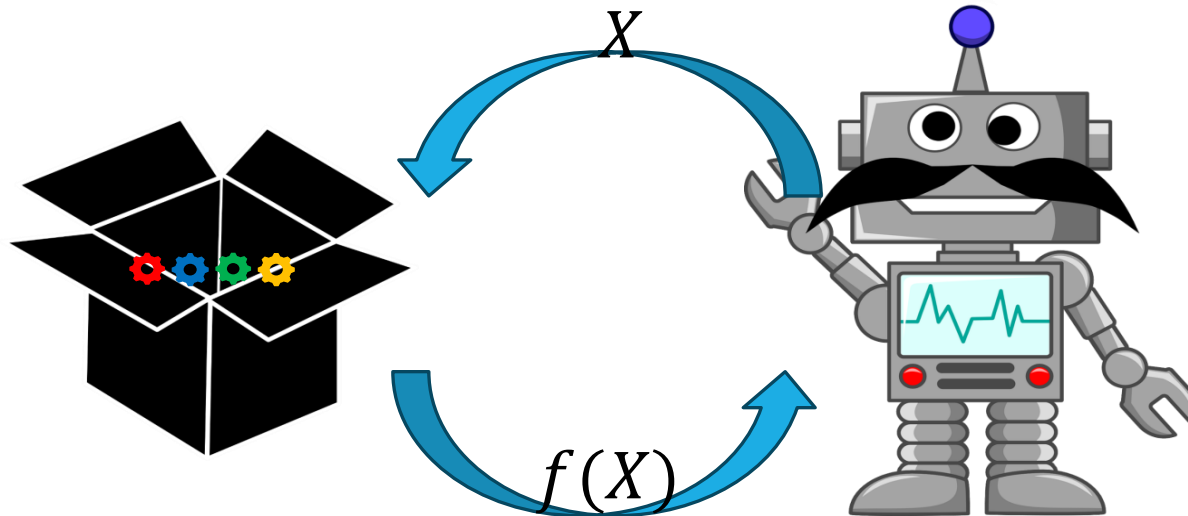
Indicators to measure performance, where which indicator is chosen is key

So, no direct equivalent to best-so-far

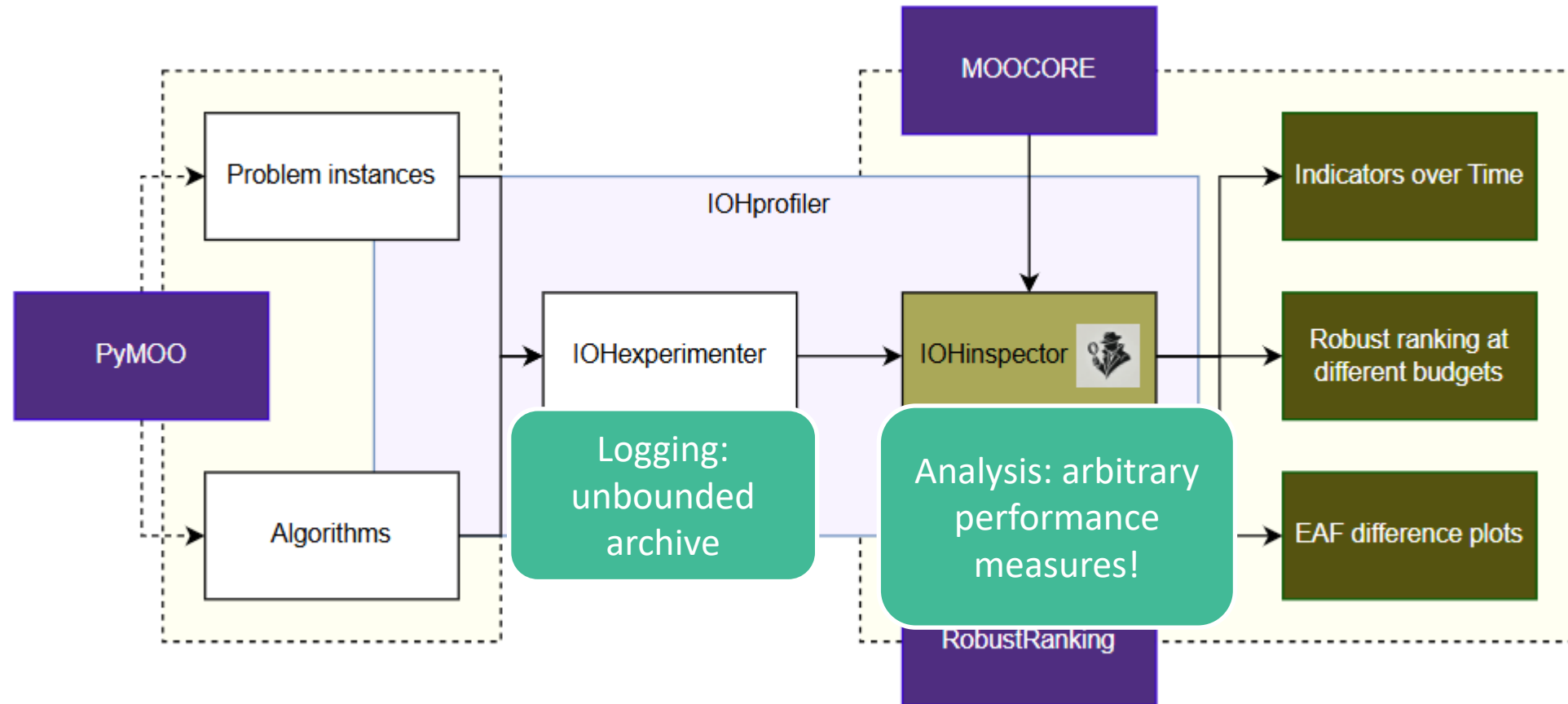
But, separation between problem and algorithm not impossible

What to log? All (or all non-dominated) solutions

Allows for later recovery of any performance metric, which can subsequently be processed in an anytime manner



Example MOO Setup: IOHInspector



Example: Comparing Population Sizes

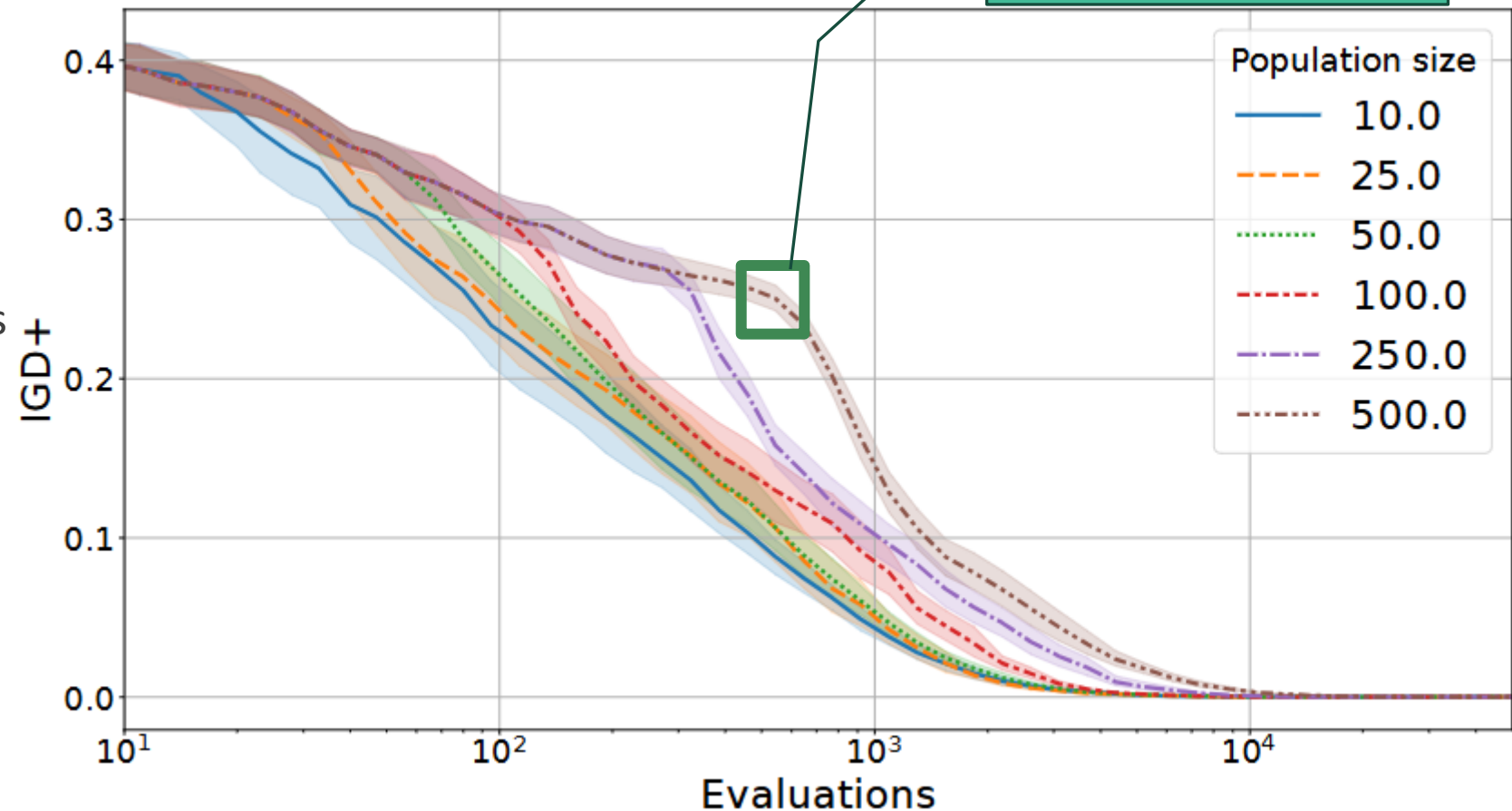
Analysis independent from used algorithm settings

Not limited to fixed solution set size

Can swap out indicators, reference sets / points ...

All without re-running experiments

Validity of analysis depends on goal of comparison and how the algorithm is considered

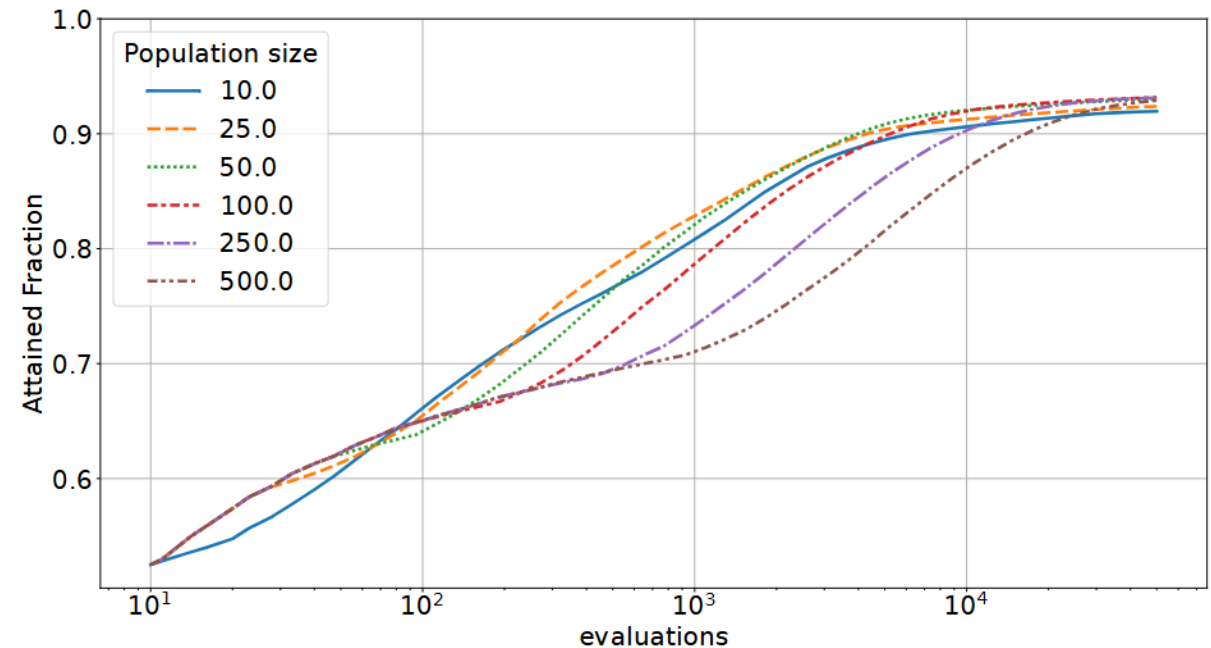


Multi-Objective Anytime Performance

From the logged set of solutions, we can reconstruct performance measures

Need to decide on several aspects:

- Should there be a fixed solution set size?
 - If yes, how to select from the larger set
 - If no, are the comparisons of different sizes fair w.r.t. the chosen indicator?
- Should solutions discarded by the algorithm be considered?
 - If yes, could this 'mislead' our analysis?
 - If no, is it still possible to have a separation between algorithm and problem in the experimental design?
- Is it fair to compare algorithms on metric X when algorithm A used X internally and B didn't?



Algorithm contribution to SOTA

No need to limit analysis to an 'overall' performance

Difference between generalist and specialist algorithms

Especially when linked with e.g. algorithm selection

Judge algorithm by contribution to SOTA, rather than average performance

Example: SHAP contributions to algorithm portfolios



Teaching benchmarking practices

Benchmarking is important step in understanding optimization algorithms

Should be accessible, especially to students

Easy-to-use examples on simple algorithms and problems

No re-inventing of the wheel in every assignment

Some examples using IOHprofiler in courses at Leiden University:

- Structure of objective functions leads to simple connections between multiple assignments (swapping objective doesn't break algorithms)
- Web-interface allows non-programmers to participate in data analysis
- Standardized data allows for easy competitive elements, e.g. bonus for best-performing algorithm on certain criteria



Hosting competitions

Competitions are a common form of benchmarking

Large potential for collecting useful performance information

Integrate competitions with existing frameworks to make entering multiple competitions straightforward

For IOHprofiler:

- Hosting our own MA-BBOB competition
- Facilitate the Submodular Optimization competition
- Interface for the GNBG-II competition



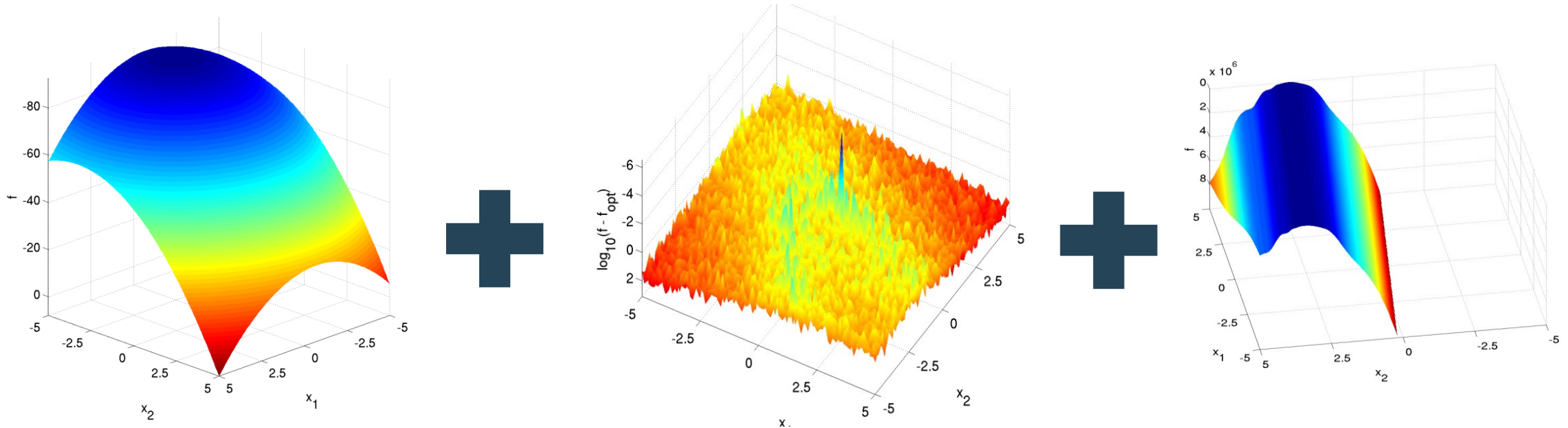
Problem Generators

Selecting suitable problems with precise characteristics can be difficult

In meta-algorithmic context, often need a lot of performance data (train/test splits)

Problem generators can be useful to create dense instance spaces

Example: MA-BBOB: affine combinations of existing BBOB functions



Current limitations of (benchmarking) software

Noiseless, unconstrained optimization is relatively well-covered

Other challenges not as much, for example

- Noisy optimization: Recommended point vs best-so-far
- Problem domains: mixed, hierarchical, variable-length,... not often covered
- Other representations: Genetic Programming,...

Interoperability between tools is still limited

Simplify integrations with meta-algorithmics, landscape analysis.



Summary: Advantages of Data Collection

Collecting performance trajectories allows for separation between experimentation and analysis

Flexibility in types of performance measures and visualizations

Promote re-use of data for different purposes

Towards an accessible repository for experimental data: OpenOpt!



Acknowledgment

We acknowledge funding by the European Union (ERC, ``dynaBBO'', grant no. 101125586). Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union or the European Research Council Executive Agency. Neither the European Union nor the granting authority can be held responsible for them.



**Funded by
the European Union**



European Research Council
Established by the European Commission